

Constant-Size zk-SNARKs in ROM from Falsifiable Assumptions

June 4, 2025

Helger Lipmaa¹[0000–0001–8393–6821], Roberto Parisella²[0009–0007–2241–801X],
and Janno Siim²[0000–0001–5824–7215]

¹ University of Tartu, Tartu, Estonia (helger.lipmaa@gmail.com)

² Simula UiB, Bergen, Norway (jannosiim@gmail.com)

Abstract. We prove that the seminal KZG polynomial commitment scheme (PCS) is black-box extractable under a simple falsifiable assumption ARSDH. To create an interactive argument, we construct a compiler that combines a black-box extractable non-interactive PCS and a polynomial IOP (PIOP). The compiler incurs a minor cost per every committed polynomial. Applying the Fiat-Shamir transformation, we obtain slightly less efficient variants of well-known PIOP-based zk-SNARKs, such as Plonk, that are knowledge-sound in the ROM under the ARSDH assumption. Importantly, there is no need for idealized group models or knowledge assumptions. This results in the first known zk-SNARKs in the ROM from falsifiable assumptions with both an efficient prover and constant-size argument.

Keywords: Black-box knowledge-soundness · polynomial commitment scheme · polynomial IOP · witness-extended emulation · zk-SNARKs

1 Introduction

Zero-knowledge Succinct Arguments of Knowledge (zk-SNARKs) allow to give a short proof of computational statements without leaking any information besides the truth of the statements. Especially in the blockchain world, efficient zk-SNARKs have found wide-scale use [BCG⁺14, KMS⁺16, Sta21] and thus are of great practical importance. Many recent zk-SNARKs are based on a combination of a polynomial commitment scheme (PCS, [KZG10]) and an information-theoretically secure non-succinct proof system like polynomial IOP [BFS20] (interactive oracle proof).

A PCS allows the prover to make a short commitment to a polynomial and later open it at a point chosen by the verifier. The very first PCS, KZG [KZG10], uses one group element for commitment and opening and two pairing operations for verification. KZG is also additively homomorphic and efficiently batchable [KZG10, TAB⁺20, BDFG20], making it ideal for zk-SNARKs with multiple PCS openings. On the negative side, it lacks a transparent setup, meaning that the public key cannot be generated from a public source of randomness. However,

the public key is updatable, making it possible to generate it in a distributed way by different parties making sequential updates to the public key. The public key is secure if at least one honest party contributes with an update. Thus, KZG is preferred for *communication* and *verifier*-efficient updatable and universal zk-SNARKs such as [GWC19, CHM⁺20, RZ21, CFF⁺21, LSZ22]. Universal means that the public key of the zk-SNARK, also known as the structured reference string (SRS), can be reused for many different relations that one wants to prove (but up to some relation size bound).

As mentioned, many recent works in communication-efficient updatable and universal zk-SNARKs start by constructing an information-theoretically secure proof system in some idealized model. For example, DARK [BFS20] uses polynomial IOP, Marlin [CHM⁺20] uses Algebraic Holographic Proof (AHP), Lunar [CFF⁺21] and Basilisk [RZ21] use Polynomial Holographic IOP (PHP), and Plonk [GWC19] uses idealized low-degree protocols. In all such information-theoretic models is that the prover sends polynomial oracles to the verifier and the verifier can make queries to the oracles. One can transform information-theoretic proofs into succinct SNARKs using an extractable PCS. The prover commits to each polynomial oracle and opens commitments at queries chosen by the verifier. Vampire [LSZ22] is a recent exception to this paradigm, giving a direct proof in AGM.

The knowledge-soundness of PCS-based zk-SNARKs relies on PCS’s extractability: given that the prover succeeds in convincing the verifier after sending a polynomial commitment and an opening, there exists an extractor that can efficiently extract a committed polynomial that is consistent with the commitment. Unfortunately, it is only known [CHM⁺20] how to prove that KZG is extractable under knowledge assumptions or in idealized models like the generic group model (GGM) and the algebraic group model (AGM, [FKL18]). For brevity, we use the acronym IGM (idealized group model) to denote any of the GGM, the AGM, or just knowledge assumptions. Hence, the knowledge-soundness of known KZG-based zk-SNARKs relies indirectly on the IGM.

On top of that, the same zk-SNARKs use the Fiat-Shamir transform, which means that they additionally rely on another strong idealization, the random oracle model (ROM). We end up in a highly undesirable situation, where efficient zk-SNARKs used in practice depend on two different idealized models. Both models, the IGM and the ROM, are known to be uninstantiable, with several papers attacking either separately. The AGM and GGM were intensively cryptanalyzed in 2022 [Zha22, ZZK22]. Zhandry and Zhang [ZZ23] recently showed that the ROM is strictly milder heuristic than Shoup’s formalization of GGM [Sho97]. Knowledge assumptions, formalized as extractable one-way functions, are known to be impossible for auxiliary input of unbounded polynomial length if a particular class of indistinguishability obfuscators exist [BCPR14].

Efficient non-updatable and non-universal (not based on PCSs) zk-SNARKs are known [GGPR13, PHGR13, Gro16] that use the IGM but not the ROM. The most efficient known updatable and universal zk-SNARKs [GKM⁺18] that rely on an IGM and do not use the ROM are too inefficient for practice.

While non-falsifiable assumptions are needed in the standard model [GW11,CGKS23], one can obtain zk-SNARKs for NP in the ROM from falsifiable assumptions. We know two types of verifier-efficient zk-SNARKs in the ROM from falsifiable assumptions. First, the zk-SNARK of Lai and Malavolta [LM19] that uses Probabilistically Checkable Proofs (PCPs). Unfortunately, PCPs are inefficient [BCGT13] (e.g., the PCP proof length is at least $\Theta(N \log^3 N)$ with a large constant, where N is the witness size), and thus this solution only has a theoretical value. In addition, PCP-based zk-SNARKs are based on inefficient arithmetizations (mathematical representation of a relation). Second, one can combine a verifier-efficient PCS in the ROM from falsifiable assumptions with a polynomial IOP. Unfortunately, the known PCSs do not result in constant-size zk-SNARKs. For example, Dory [Lee21] and [BMM⁺21] have an efficient prover but log-length arguments.

It is unknown if zk-SNARKs with an efficient, say $O_\lambda(N \log N)^3$, prover and $O_\lambda(1)$ argument size are possible in the ROM under falsifiable assumption. Constructing one — especially with a small constant in both $O_\lambda(\cdot)$ -s — is an important open problem: it allows one to base zk-SNARKs on more secure foundations without sacrificing efficiency. Ideally, one would like to prove an already well-established zk-SNARK (such as Plonk) to be secure under weaker assumptions since an efficient arithmetization (say, R1CS or Plonk’s arithmetization) and much infrastructure already accompanies it.

We propose the following two questions.

- **Theoretical:** Does a zk-SNARK with $O_\lambda(1)$ proof size and an efficient prover/verifier exist that is secure in the ROM under falsifiable assumptions?
- **Practical:** Is the KZG-based Plonk secure in the ROM under falsifiable assumptions?

We answer positively to the first question and achieve notable progress on the second.

Our Contributions. In this work, we consider non-interactive (univariate) PCSs over a field $\mathbb{F}$ where both the commitment and opening phases are non-interactive (a single message). The KZG [KZG10] is a prime example. Assume n is a degree bound on committed polynomials. Although the commitment and opening phases are non-interactive, as a whole a non-interactive PCS can be viewed as a three-round protocol:

1. the prover sends a commitment C to some polynomial $f(X) \in \mathbb{F}[X]$ of degree at most n ,
2. the verifier responds with an evaluation point $\alpha \in \mathbb{F}$, and
3. the prover sends $\eta = f(\alpha)$ and an opening proof π .

Note that some other PCSs, such as the one in DARK [BFS20], have an interactive opening phase. In the following, unless specified otherwise, we mean a non-interactive PCS when we write PCS.

³ Here, $O_\lambda(\cdot)$ is the common “Big O” notation, but we ignore $\text{poly}(\lambda)$ factors.

We define computational k -special-soundness and black-box extractability for PCS by following the definitions of k -special-soundness and black-box extractability for proof systems.

More precisely, a PCS satisfies computational $(n + 1)$ -special-soundness if there exists an efficient extractor, such that: if an efficient adversary produces $n + 1$ accepting PCS transcripts $(C, \alpha_j, \eta_j, \pi_j)$ with the same commitment C but distinct evaluation points α_j , then the extractor extracts a polynomial f of degree at most n , that is consistent with the commitment C and satisfies $f(\alpha_j) = \eta_j$ for all $j = 1, \dots, n + 1$. We prove that KZG is computationally $(n + 1)$ -special-sound under a new but falsifiable and standard-looking assumption ARSDH (*Adaptive Rational Strong Diffie-Hellman*). ARSDH is an adaptive variant of the known assumption RSDH (*Rational Strong Diffie-Hellman*) of González and Ràfols [GR19]. Interestingly, our special-soundness reduction uses the techniques of [TAB⁺20] to combine $n + 1$ openings to a single batch opening.

We prove that ARSDH is secure in the AGM with oblivious sampling (AGMOS) [LPS23]. AGMOS is a more realistic version of AGM that additionally allows the adversary to sample group elements without knowing their discrete logarithms. We emphasize that our special-soundness proof does not depend on the AGMOS; we use the AGMOS only as a sanity check for the *falsifiable* ARSDH assumption. We also prove that ARSDH implies the strong Diffie-Hellman assumption and thus implies the evaluation binding of KZG (it is difficult to open a polynomial commitment to two different values at the same evaluation point).

Next, we define black-box extractability for non-interactive PCS. A non-interactive PCS is black-box extractable if there exists an expected probabilistic polynomial time (PPT) black-box extractor Ext_{bb} , such that for each efficient adversary $(\mathcal{A}, \text{P}^*)$, where $\mathcal{A}$ produces commitments and P^* produces openings: given a maliciously generated transcript $\text{tr}_0 = (C, \alpha_0, \eta_0, \pi_0)$, for randomly sampled α_0 , and an oracle access to P^* , the extractor outputs a polynomial f , such that if the PCS verifier accepts tr_0 then f agrees with tr_0 (C is a commitment of f and $f(\alpha_0) = \eta_0$). The evaluation point α_0 is sampled uniformly and independently from C from some super-polynomially large subset⁴ $\mathcal{F}$ of $\mathbb{F}$.

We prove that every computationally $(n + 1)$ -special-sound non-interactive PCS is black-box extractable. Let ck be a commitment key. The black-box extractor $\text{Ext}_{\text{bb}}^{\text{P}^*}(\text{ck}, \text{tr}_0)$ rejects if the PCS verifier V rejects tr_0 . Otherwise, Ext_{bb} invokes another extractor $\text{Ext}_{\text{rw}}^{\text{P}^*}(\text{ck}, \text{tr}_0)$ that outputs n transcripts $\mathbf{tr} = (\text{tr}_1, \dots, \text{tr}_n)$. Ext_{bb} rejects if V rejects tr_j for some $j \geq 1$. Otherwise, Ext_{bb} uses the special-soundness extractor (that exists since PCS is special sound) on input $\text{tr}_0 \parallel \mathbf{tr}$ to extract f . One complication is that Ext_{rw} (described in the next paragraph) is an expected PPT algorithm. To prove that Ext_{bb} works, we need to define a special-soundness reduction $\mathcal{A}_{\text{ss}}$, which internally runs Ext_{rw} . However, special-soundness holds against strict PPT adversaries. To resolve this mismatch, we prove a general result that if a falsifiable security game [Nao03, GW11] (one

⁴ In zk-SNARKs in the literature, one can have say $\mathcal{F} = \mathbb{F}$, $\mathcal{F} = \mathbb{F}^*$, $\mathcal{F} = \mathbb{F} \setminus \mathbb{H}$ for a multiplicative subgroup of $\mathbb{H}$, etc.

where an efficient challenger interacts with an adversary) holds respect to any strict PPT adversary, then it also holds against any expected PPT adversary.

The most challenging part of the reduction is the rewinding extractor Ext_{rw} . Ext_{rw} has the following goal: given that the adversary can produce a single transcript $\text{tr}_0 = (C, \alpha_0, \dots)$, accepted by the PCS verifier, Ext_{rw} produces with an overwhelming probability n more accepting transcripts that share the same commitment but have pairwise distinct second elements α_j . For a fixed commitment C , Ext_{rw} runs P^* with distinct random evaluation points α until it obtains n accepting transcripts. The proof that Ext_{rw} produces a correct output with an overwhelming probability in expected PPT is technical but similar to proofs of other such extractors (especially [ACK21]).

Given the above, we can conclude that the KZG PCS is black-box extractable under the ARSDH assumption.

Compiler. Following earlier works like [CHM⁺20, CFF⁺21, BFS20], we present a general compiler, which combines a non-interactive PCS and a polynomial IOP [BFS20] into an interactive argument.

Polynomial IOP [BFS20] is an idealized information-theoretic proof system, where in each round of a protocol, the prover sends a polynomial oracle to the verifier, and the verifier replies with a challenge. The verifier can also query the oracles. Query points are revealed to the prover, who can use them to construct polynomial oracles of the subsequent rounds. For example, in Plonk [GWC19], several polynomials are opened at $\mathfrak{z} \leftarrow_{\$} \mathbb{F}$, and one polynomial is opened at $\omega \cdot \mathfrak{z}$, where ω is a known value (a primitive root of unity). In Vampire [LSZ22], some polynomials are opened at whole (known) subgroups. Finally, the verifier either rejects or accepts the proof based on the responses from the oracles.

Bünz et al. (DARK, [BFS20]) prove that when combining a polynomial IOP with a knowledge sound PCS (the prover knows the committed polynomial), one obtains an interactive argument system for the same relation. In DARK, the opening phase is an interactive argument for proving knowledge of the committed polynomial. This is a crucial difference with our work, where commitment and opening phases are non-interactive.

Our compiler follows the execution of the polynomial IOP protocol but with the following differences. First, when the polynomial IOP prover sends a polynomial oracle f , the argument's prover sends a commitment of f , the argument's verifier responds with $\chi \leftarrow_{\$} \mathcal{F}$ ($\mathcal{F}$ is some superpolynomial size set), and the prover opens the commitment at the point χ .⁵ Second, when the polynomial IOP verifier wants to query one of its oracles g , it sends the query point α to the prover, which then opens the commitment of g at point α .

Our compiler's security relies on the knowledge soundness of the polynomial IOP and on the PCS's black-box extractability and evaluation binding.

⁵ Alternatively, one can define the following version of the KZG commitment secure in the ROM. The commitment phase is (C, η, π) , where $\chi = H(C)$, and H is a random oracle. This guarantees extractability. Now, in the opening phase one can use arbitrary evaluation points α' that do not have to be uniformly random.

We prove that the compiled argument satisfies witness-extended emulation (WEE, [Lin01]). Intuitively, given an adversary that breaks witness-extended emulation of the argument, we can construct an adversary that breaks the knowledge soundness of the polynomial IOP. When the argument’s prover (potentially malicious) outputs a commitment and successfully opens it at an extraction point χ , the reduction extracts the polynomial f and sends it to the polynomial IOP verifier. The evaluation binding property guarantees that the queries that the polynomial IOP verifier makes to f are consistent with the evaluations that the argument’s prover outputs. Otherwise, if $f(\alpha) \neq \eta$ for some claimed evaluation η outputted by the argument’s prover, we get a collision (contradicting evaluation binding).

We can use the polynomial IOP of popular zk-SNARKs like [GWC19] and Marlin [CHM⁺20] and apply our compiler with KZG. Our compiler adds only a small overhead. First, our compiler is for polynomial IOP, so it works only for polynomial IOP variants of such zk-SNARKs. Second, we must open each committed polynomial at one more random point. This results in a public coin argument, secure under the falsifiable ARSDH assumption, with $O_\lambda(1)$ proof size, $O_\lambda(N \log N)$ prover’s computation time, $O_\lambda(|\mathbb{x}| + \log N)$ verifier’s computation time, and $O_\lambda(N)$ length structured reference string (SRS). Here, N denotes the circuit size representing the proven relation, and $|\mathbb{x}|$ is the statement size. Importantly, the compiled variant will be universal and updatable. Some care must be taken with zero-knowledge since we introduce additional queries to the polynomial IOPs. However, this can be adjusted by introducing additional randomness to the polynomials, which adds minor extra cost. We did not try to find a generic approach as it seems to be better to handle it in a case-by-case basis.

To our knowledge, this is the first argument system with a constant proof size and an efficient prover and verifier in the ROM under falsifiable assumptions. After applying the Fiat-Shamir heuristic, we obtain a SNARK that is secure in the ROM under the ARSDH assumption. This answers the first question we proposed in the introduction.

As for the second question (is Plonk secure in the ROM under falsifiable assumptions?), we obtain partial success. Our compiler outputs a version of Plonk, which is less efficient by a small constant factor. For each polynomial committed by the prover, we will have an additional opening proof (1 group element) and an evaluation of the polynomial (1 field element). Note that the evaluation point χ does not add to the communication size since the prover and the verifier compute χ locally using a hash function when applying the Fiat-Shamir heuristic. Plonk’s (or Marlin’s) efficiency also relies on several small optimizations: batching of commitment openings and the so-called Maller’s trick [GWC19]. We leave it an open question if our notion of extractability is sufficient to prove the security of such optimizations. We also leave it as an open question if it is possible to tightly reduce the soundness of our SNARK to the underlying ARSDH assumption.

Extractability in AGM(OS). The extractability of the KZG PCS in the AGM was proven in [CHM⁺20]. They also proposed a slightly less efficient version of KZG, where a knowledge component accompanies each commitment. In that case, extractability is possible under a knowledge assumption, but the commitment length is two group elements instead of one (i.e., the extra cost is one group element). When using our extraction technique, we get purely rewinding-based extractability with the cost of one group element and one field element.

Lipmaa et al. [LPS23] recently proposed AGMOS (AGM with oblivious sampling). AGMOS is a more realistic variant of the AGM [FKL18] that gives the adversary additional power to obliviously sample group elements without knowing their discrete logarithms. Moreover, [LPS23] pointed out that researchers use KZG extractability in two different senses. In many papers (e.g., [CHM⁺20]), KZG extractability means extracting the polynomial after both the commitment and opening phase. In other papers, it means the ability of polynomial extraction after the commitment phase only. For example, Lunar [CFF⁺21] and Plonk [GWC19] assume that one can extract the polynomial directly after the commitment phase; however, this results [LPS23] in a spurious knowledge assumption that is secure in the AGM but insecure in the standard model.

Lipmaa et al. [LPS23] analyzed KZG in AGMOS and provided an AGMOS proof that f can be extracted from the KZG commitment C and an acceptable opening (η, π) of the commitment at any evaluation point α . In particular, α does not have to be sampled from a set of superpolynomial size (one can pick $\alpha = 0$, for example). We work without using any idealized group model like AGM(OS), but we pay by having the random evaluation requirement.

2 Preliminaries

Let λ denote the security parameter. PPT stands for probabilistic polynomial time, and DPT for deterministic polynomial time. We say *expected PPT* when referring to probabilistic Turing machines whose expected running time is bounded by a polynomial in the security parameter. All adversaries are implicitly assumed to be non-uniform. Other algorithms are uniform. $\mathbb{F}$ is a finite field of prime order p ; let $\mathbb{F}^* := \mathbb{F} \setminus \{0\}$. We denote by $\mathbb{F}_{\leq n}[X]$ the ring of univariate polynomials with variable X over $\mathbb{F}$ of degree $\leq n$. When a is uniformly sampled from a set A , we write $a \leftarrow_{\$} A$. A negligible function δ is a function such that, for every polynomial f , there exists an integer N_f such that, if $\lambda > N_f$, then $|\delta(\lambda)| \leq 1/f(\lambda)$. We write $\delta(\lambda) \approx_{\lambda} 0$, when $\delta(\lambda)$ is a negligible function. By $f(\lambda) \in \text{poly}(\lambda)$, we mean that the function $f(\lambda)$ is asymptotically bounded by some polynomial.

Lagrange Interpolation. Assume n is a power of two. Let ω be the n -th primitive root of unity modulo p and let $\mathbb{H} = \langle \omega \rangle$ be the multiplicative subgroup of $\mathbb{F}$ generated by ω . (ω exists, given that $n \mid (p-1)$.)

For $j \in [1, n]$, let $\ell_j(X)$ be the j -th Lagrange polynomial, that is, the unique degree $n-1$ polynomial, such that $\ell_j(\omega^{j-1}) = 1$ and $\ell_i(\omega^{j-1}) = 0$ for $i \neq j$. It is well known that $\ell_j(X) = ((X^n - 1)\omega^{j-1}) / (n(X - \omega^{j-1}))$ for $X \neq \omega^{j-1}$.

Bilinear Groups. A bilinear group generator $\text{Pgen}(1^\lambda)$ returns $\mathbf{p} = (p, \mathbb{G}_1, \mathbb{G}_2, \mathbb{G}_T, \hat{e}, [1]_1, [1]_2)$, where $\mathbb{G}_1, \mathbb{G}_2$, and $\mathbb{G}_T$ are additive cyclic (thus, abelian) groups of prime order p , $\hat{e} : \mathbb{G}_1 \times \mathbb{G}_2 \rightarrow \mathbb{G}_T$ is a non-degenerate efficiently computable bilinear pairing, and $[1]_\iota$ is a fixed generator of $\mathbb{G}_\iota$. While $[1]_\iota$ is a part of $\mathbf{p}$, for the sake of clarity, we often give it as an explicit input to different algorithms. The bilinear pairing is of Type-3, that is, there is no efficient isomorphism between $\mathbb{G}_1$ and $\mathbb{G}_2$. We use the standard bracket notation, that is, for $\iota \in \{1, 2, T\}$ and $x \in \mathbb{Z}_p$, we write $[x]_\iota$ to denote $x[1]_\iota$. We denote $\hat{e}([x]_1, [y]_2)$ by $[x]_1 \bullet [y]_2$ and assume $[1]_T = [1]_1 \bullet [1]_2$. Thus, $[x]_1 \bullet [y]_2 = [xy]_T$ for any $x, y \in \mathbb{F}$.

Assumptions. Let $d_1(\lambda), d_2(\lambda) \in \text{poly}(\lambda)$. Pgen is (d_1, d_2) -PDL (*Power Discrete Logarithm [Lip12]*) secure if for any PPT $\mathcal{A}$, $\text{Adv}_{d_1, d_2, \text{Pgen}, \mathcal{A}}^{\text{pdl}}(\lambda) :=$

$$\Pr \left[\mathcal{A}(\mathbf{p}, [(\sigma^i)_{i=0}^{d_1}]_1, [(\sigma^i)_{i=0}^{d_2}]_2) = \sigma \mid \mathbf{p} \leftarrow \text{Pgen}(1^\lambda); \sigma \leftarrow_{\$} \mathbb{F}^* \right] \approx_{\lambda} 0 .$$

Let $d(\lambda) \in \text{poly}(\lambda)$. Pgen is d -SDH (*Strong Diffie-Hellman, [BB08]*) secure in $\mathbb{G}_1$, if for any PPT $\mathcal{A}$, $\text{Adv}_{\text{Pgen}, 1, d, \mathcal{A}}^{\text{sdh}}(\lambda) :=$

$$\Pr \left[\begin{array}{c} \sigma + c \neq 0 \wedge \\ [\varphi]_1 = \frac{1}{\sigma + c} \cdot [1]_1 \end{array} \mid \begin{array}{c} \mathbf{p} \leftarrow \text{Pgen}(1^\lambda); \sigma \leftarrow_{\$} \mathbb{Z}_p; \\ (c, [\varphi]_1) \leftarrow \mathcal{A}(\mathbf{p}, ([\sigma^i]_1)_{i=0}^d, [1, \sigma]_2) \end{array} \right] \approx_{\lambda} 0 .$$

A security game is *falsifiable* if it can be expressed as an interaction between an efficient challenger and an adversary [Nao03, GW11]. In Section A, we show that if a falsifiable game is secure respect to any strict PPT adversary, it is also secure respect to any expected PPT adversary. We use that result in several of our reductions.

2.1 Polynomial Commitment Schemes

In a *non-interactive* (univariate) polynomial commitment scheme (PCS, [KZG10]), the prover commits to a polynomial $f \in \mathbb{F}_{\leq n}[X]$ and later opens it to $f(\alpha)$ for $\alpha \in \mathbb{F}$ chosen by the verifier. A (non-randomized) non-interactive polynomial commitment scheme [KZG10] consists of the following algorithms:

Setup $\text{Pgen}(1^\lambda) \mapsto \mathbf{p}$: Given 1^λ , return system parameters $\mathbf{p}$.

Commitment key generation $\text{KGen}(\mathbf{p}, n) \mapsto (\text{ck}, \text{tk})$: Given a system parameter $\mathbf{p}$ and an upperbound n on the polynomial degree, return (ck, tk) , where ck is the commitment key and tk is the trapdoor. We assume ck implicitly contains $\mathbf{p}$. In the context of this paper, we do not use the trapdoor.

Commitment $\text{Com}(\text{ck}, f) \mapsto C$: Given a commitment key ck and a polynomial $f \in \mathbb{F}_{\leq n}[X]$, return a commitment C .

Opening $\text{Open}(\text{ck}, C, \alpha, f) \mapsto (\eta, \pi)$: Given a commitment key ck , a commitment C , an evaluation point $\alpha \in \mathbb{F}$, and a polynomial $f \in \mathbb{F}_{\leq n}[X]$, return (η, π) , where $\eta \leftarrow f(\alpha)$ and π is an evaluation proof.

Verification $V(\text{ck}, C, \alpha, \eta, \pi) \mapsto \{0, 1\}$: Given a commitment key ck , a commitment C , an evaluation point α , a purported evaluation $\eta = ? f(\alpha)$, and an evaluation proof π , return 1 (accept) or 0 (reject).

Thus, in a non-interactive PCS, both the commitment and the opening are non-interactive (it only consists of a single message that can be verified non-interactively). In an *interactive PCS*, either opening or verification (or both) is an interactive protocol. Most of the known PCSs, like FRI [BBHR18], Bulletproofs [BBB⁺18], and DARK [BFS20], are interactive. Univariate KZG [KZG10] and multivariate PST [PST13] are two well-known non-interactive PCSs.

A non-interactive PCS PC is complete, if for any λ , $\mathbf{p} \leftarrow \text{Pgen}(1^\lambda)$, $n \in \text{poly}(\lambda)$, $\alpha \in \mathbb{F}$, and $f \in \mathbb{F}_{\leq n}[X]$,

$$\Pr [V(\text{ck}, C, \alpha, \eta, \pi) = 1 \mid (\text{ck}, \text{tk}) \leftarrow \text{KGen}(\mathbf{p}, n); (\eta, \pi) \leftarrow \text{Open}(\text{ck}, C, \alpha, f)] = 1.$$

Definition 1. A non-interactive polynomial commitment scheme PC is (non-black-box) extractable for Pgen , if for any $n \in \text{poly}(\lambda)$, and PPT adversary $\mathcal{A}$, there exists a PPT extractor $\text{Ext}_{\mathcal{A}}$, such that $\text{Adv}_{\text{Pgen}, \text{PC}, n, \mathcal{A}, \text{Ext}_{\mathcal{A}}}^{\text{ext}}(\lambda) :=$

$$\Pr \left[\begin{array}{l} V(\text{ck}, C, \alpha, \eta, \pi) = 1 \wedge \\ \left(C \neq \text{Com}(f(X)) \vee \right. \\ \left. \deg f > n \vee f(\alpha) \neq \eta \right) \end{array} \mid \begin{array}{l} \mathbf{p} \leftarrow \text{Pgen}(1^\lambda); (\text{ck}, \text{tk}) \leftarrow \text{KGen}(\mathbf{p}, n); \\ r \leftarrow \text{RND}_\lambda(\mathcal{A}); (C, \alpha, \eta, \pi) \leftarrow \mathcal{A}(\text{ck}; r); \\ f(X) \leftarrow \text{Ext}_{\mathcal{A}}(\text{ck}; r) \end{array} \right] \approx_\lambda 0,$$

where $r \leftarrow \text{RND}_\lambda(\mathcal{A})$ denotes sampling random coins for $\mathcal{A}$.

A weaker property is evaluation binding [KZG10], which does not include extractability, but disallows opening the same evaluation point to different evaluations.

Definition 2. A polynomial commitment scheme PC is evaluation binding for Pgen , if for any $n \in \text{poly}(\lambda)$, and PPT adversary $\mathcal{A}$, $\text{Adv}_{\text{Pgen}, \text{PC}, n, \mathcal{A}}^{\text{evbind}}(\lambda) :=$

$$\Pr \left[\begin{array}{l} V(\text{ck}, C, \alpha, \eta, \pi) = 1 \wedge \\ V(\text{ck}, C, \alpha, \eta', \pi') = 1 \wedge \eta \neq \eta' \end{array} \mid \begin{array}{l} \mathbf{p} \leftarrow \text{Pgen}(1^\lambda); (\text{ck}, \text{tk}) \leftarrow \text{KGen}(\mathbf{p}, n); \\ (C, \alpha, \eta, \pi, \eta', \pi') \leftarrow \mathcal{A}(\mathbf{p}, \text{ck}) \end{array} \right] \approx_\lambda 0.$$

The seminal (non-randomized) KZG [KZG10] polynomial commitment scheme is defined as follows:

KZG.Pgen(λ): return $\mathbf{p} \leftarrow \text{Pgen}(1^\lambda)$.

KZG.KGen($\mathbf{p}, n$): $\text{tk} = \sigma \leftarrow \mathbb{Z}_p^*$; $\text{ck} \leftarrow (\mathbf{p}, [(\sigma^i)_{i=0}^n]_1, [1, \sigma]_2)$; return (ck, tk) .

KZG.Com(ck, f): return $C \leftarrow [f(\sigma)]_1 = \sum_{j=0}^n f_j [\sigma^j]_1$.

KZG.Open(ck, C, α, f): $\eta \leftarrow f(\alpha)$; $\varphi(X) \leftarrow (f(X) - \eta)/(X - \alpha)$; $\pi \leftarrow [\varphi(\sigma)]_1$; return (η, π) .

KZG.V($\text{ck}, C, \alpha, \eta, \pi$): Return 1 iff $(C - \eta[1]_1) \bullet [1]_2 = \pi \bullet [\sigma - \alpha]_2$.

KZG's security is based on the fact that $(X - \alpha) \mid (f(X) - \eta) \Leftrightarrow f(\alpha) = \eta$. KZG is evaluation binding under the n -SDH assumption [KZG10] and non-black-box extractable in the AGM [CHM⁺20] under the PDL assumption and in AGMOS under the PDL and TOFR (see Section C) assumptions.

2.2 Succinct Zero-Knowledge Arguments

Indexed Relation. We consider relations that depend on the pairing description $\mathbf{p} \leftarrow \text{Pgen}(1^\lambda)$. An indexed relation $\mathcal{R}_{\mathbf{p}}$ is a relation of triples $(\mathbf{i}, \mathbf{x}, \mathbf{w})$, where $\mathbf{i}$ is an index (e.g., an arithmetic circuit), $\mathbf{x}$ is a statement (e.g., a public input to the circuit) and $\mathbf{w}$ is an NP-witness (e.g., a private input to the circuit) for the language $\mathcal{L}(\mathcal{R}_{\mathbf{p}}) := \{(\mathbf{i}, \mathbf{x}) : (\mathbf{i}, \mathbf{x}, \mathbf{w}) \in \mathcal{R}_{\mathbf{p}}\}$. We denote $\mathbb{I}(\mathcal{R}_{\mathbf{p}}) = \{\mathbf{i} : (\mathbf{i}, \mathbf{x}, \mathbf{w}) \in \mathcal{R}_{\mathbf{p}}\}$. We also consider subrelations $\mathcal{R}_{\mathbf{p},n} \subset \mathcal{R}_{\mathbf{p}}$, where the size of index $\mathbf{i}$ is bounded by n .

Argument System. Groth et al. [GKM⁺18] introduced the notion of (pre-processing) zk-SNARKs with specializable universal structured reference string (SRS). This notion formalizes the idea that the key generation for $\mathcal{R}_{\mathbf{p},n}$ can be seen as the sequential combination of two steps. First, a probabilistic algorithm KGen generates a SRS for $\mathcal{R}_{\mathbf{p},n}$ (e.g., for satisfiability of any circuit with $\leq n$ gates) and second, a deterministic algorithm Derive specializes the universal SRS into one for a specific $\mathbf{i} \in \mathbb{I}(\mathcal{R}_{\mathbf{p},n})$ (e.g., for a fixed circuit with $\leq n$ gates).

Let $(\mathbf{P}, \mathbf{V})$ be a pair of interactive algorithms where $\mathbf{V}$ outputs the final message (typically either 0 or 1, unless $\mathbf{V}$ is malicious). We denote by $\text{tr} \leftarrow \langle \mathbf{P}(x), \mathbf{V}(y) \rangle$ the protocol transcript when $\mathbf{P}$ gets an input x and $\mathbf{V}$ gets an input y . For simplicity we sometimes write $\langle \mathbf{P}(x), \mathbf{V}(y) \rangle = b$ when we compare $\mathbf{V}$'s last message to b .

A *succinct zero-knowledge argument system* $\Pi = (\text{Pgen}, \text{KGen}, \text{Derive}, \mathbf{P}, \mathbf{V})$ with *specializable universal SRS for a relation family* $(\text{Pgen}, \{\mathcal{R}_{\mathbf{p},n}\}_{\mathbf{p} \in \text{range}(\text{Pgen}), n \in \mathbb{N}})$ consists of the following algorithms.

Setup: Given 1^λ , return system parameters $\mathbf{p} \leftarrow \text{Pgen}(1^\lambda)$.

Universal SRS Generation: a probabilistic algorithm $\text{KGen}(\mathbf{p}, n) \rightarrow (\mathbf{srs}, \mathbf{td}_{\mathbf{srs}})$ that takes as input public parameters $\mathbf{p}$ and an upper bound n on the index size, and outputs $\mathbf{srs}$ together with a trapdoor. We assume that $\mathbf{srs}$ contains $\mathbf{p}$.

SRS Specialization: a deterministic algorithm $\text{Derive}(\mathbf{srs}, \mathbf{i}) \rightarrow (\mathbf{ek}_{\mathbf{i}}, \mathbf{vk}_{\mathbf{i}})$ that takes as input a universal SRS $\mathbf{srs}$ and an index $\mathbf{i} \in \mathbb{I}(\mathcal{R}_{\mathbf{p},n})$, and outputs a specialized SRS $\mathbf{srs}_{\mathbf{i}} := (\mathbf{ek}_{\mathbf{i}}, \mathbf{vk}_{\mathbf{i}})$. Here $\mathbf{ek}_{\mathbf{i}}$ is for the prover, and $\mathbf{vk}_{\mathbf{i}}$ is for the verifier. We assume that $\mathbf{ek}_{\mathbf{i}}$ and $\mathbf{vk}_{\mathbf{i}}$ contain $\mathbf{p}$.

Prover/Verifier: a pair of interactive algorithms $\langle \mathbf{P}(\mathbf{ek}_{\mathbf{i}}, \mathbf{x}, \mathbf{w}), \mathbf{V}(\mathbf{vk}_{\mathbf{i}}, \mathbf{x}) \rangle = b$, where $\mathbf{P}$ takes a proving key $\mathbf{ek}_{\mathbf{i}}$ for an index $\mathbf{i}$, a statement $\mathbf{x}$, and a witness $\mathbf{w}$, s.t. $(\mathbf{i}, \mathbf{x}, \mathbf{w}) \in \mathcal{R}_{\mathbf{p},n}$, and $\mathbf{V}$ takes a verification key $\mathbf{vk}_{\mathbf{i}}$ for an index $\mathbf{i}$ and a statement $\mathbf{x}$, and either accepts ($b = 1$) or rejects ($b = 0$) the argument.

Π must satisfy the following four requirements.

Completeness. For all $\mathbf{p} \in \text{range}(\text{Pgen})$, $n \in \mathbb{N}$, $(\mathbf{i}, \mathbf{x}, \mathbf{w}) \in \mathcal{R}_{\mathbf{p},n}$,

$$\Pr \left[\langle \mathbf{P}(\mathbf{ek}_{\mathbf{i}}, \mathbf{x}, \mathbf{w}), \mathbf{V}(\mathbf{vk}_{\mathbf{i}}, \mathbf{x}) \rangle = 1 \mid \begin{array}{l} (\mathbf{srs}, \mathbf{td}_{\mathbf{srs}}) \leftarrow \text{KGen}(\mathbf{p}, n); \\ (\mathbf{ek}_{\mathbf{i}}, \mathbf{vk}_{\mathbf{i}}) \leftarrow \text{Derive}(\mathbf{srs}, \mathbf{i}) \end{array} \right] = 1 .$$

Witness-Extended Emulation. Π satisfies witness-extended emulation if for every DPT P^* there exists an expected polynomial time emulator Emu ⁶, such that for any λ , PPT adversary $\mathcal{A}$, PPT distinguisher $\mathcal{D}$, and $n \in \text{poly}(\lambda)$, $\text{Adv}_{\text{Pgen}, \Pi, n, P^*, \mathcal{A}, \mathcal{D}}^{\text{wee}}(\lambda) := |\varepsilon_0 - \varepsilon_1| \approx_\lambda 0$, where

$$\begin{aligned} \varepsilon_0 &:= \Pr \left[\begin{array}{l} \mathbf{i} \in \mathbb{I}(\mathcal{R}_{p,n}) \wedge \\ \mathcal{D}(\text{tr}) = 1 \end{array} \left| \begin{array}{l} \mathbf{p} \leftarrow \text{Pgen}(1^\lambda); (\mathbf{srs}, \mathbf{td}_{\mathbf{srs}}) \leftarrow \text{KGen}(\mathbf{p}, n); \\ (\mathbf{i}, \mathbf{x}, \mathbf{st}) \leftarrow \mathcal{A}(\mathbf{p}, \mathbf{srs}); (\mathbf{ek}_i, \mathbf{vk}_i) \leftarrow \text{Derive}(\mathbf{srs}, \mathbf{i}); \\ \text{tr} \leftarrow \langle P^*(\mathbf{p}, \mathbf{srs}, \mathbf{x}, \mathbf{st}), \mathbf{V}(\mathbf{vk}_i, \mathbf{x}) \rangle \end{array} \right. \right], \\ \varepsilon_1 &:= \Pr \left[\begin{array}{l} \mathbf{i} \in \mathbb{I}(\mathcal{R}_{p,n}) \wedge \\ \mathcal{D}(\text{tr}) = 1 \wedge \\ \left(\mathbf{V}_{\text{check}}(\mathbf{srs}, \text{tr}) = 1 \right) \\ \Rightarrow (\mathbf{i}, \mathbf{x}, \mathbf{w}) \in \mathcal{R} \end{array} \left| \begin{array}{l} \mathbf{p} \leftarrow \text{Pgen}(1^\lambda); (\mathbf{srs}, \mathbf{td}_{\mathbf{srs}}) \leftarrow \text{KGen}(\mathbf{p}, n); \\ (\mathbf{i}, \mathbf{x}, \mathbf{st}) \leftarrow \mathcal{A}(\mathbf{p}, \mathbf{srs}); \\ (\mathbf{ek}_i, \mathbf{vk}_i) \leftarrow \text{Derive}(\mathbf{srs}, \mathbf{i}); \\ (\text{tr}, \mathbf{w}) \leftarrow \text{Emu}^{\langle P^*(\mathbf{p}, \mathbf{srs}, \mathbf{x}, \mathbf{st}), \mathbf{V}(\mathbf{vk}_{\mathcal{R}}, \mathbf{x}) \rangle}(\mathbf{p}, \mathbf{srs}, \mathbf{x}) \end{array} \right. \right], \end{aligned}$$

where Emu has access to a transcript oracle that can be rewound to any round and run again with fresh random coins of the verifier. $\mathbf{V}_{\text{check}}(\mathbf{srs}, \text{tr})$ outputs 1 if the transcript is accepted by the verifier and 0 otherwise.

Honest Verifier Zero-Knowledge. Π is ε -statistical *honest verifier zero-knowledge* if there exists a PPT simulator Sim , s.t. for all unbounded algorithms $\mathcal{D} = (\mathcal{D}_1, \mathcal{D}_2)$, all $\mathbf{p} \in \text{range}(\text{Pgen})$, all $n \in \text{poly}(\lambda)$, $|\varepsilon_0(\lambda) - \varepsilon_1(\lambda)| \leq \varepsilon(\lambda)$, where

$$\begin{aligned} \varepsilon_0(\lambda) &:= \Pr \left[\begin{array}{l} \mathcal{D}_2(\mathbf{st}, \text{tr}) = 1 \wedge \\ \mathcal{R}_{p,n}(\mathbf{i}, \mathbf{x}, \mathbf{w}) \end{array} \left| \begin{array}{l} (\mathbf{srs}, \mathbf{td}_{\mathbf{srs}}) \leftarrow \text{KGen}(\mathbf{p}, n); (\mathbf{i}, \mathbf{x}, \mathbf{w}, \mathbf{st}) \leftarrow \mathcal{D}_1(\mathbf{srs}); \\ (\mathbf{ek}_i, \mathbf{vk}_i) \leftarrow \text{Derive}(\mathbf{srs}, \mathbf{i}); \\ \text{tr} \leftarrow \langle P(\mathbf{ek}_i, \mathbf{x}, \mathbf{w}), \mathbf{V}(\mathbf{vk}_i, \mathbf{x}) \rangle \end{array} \right. \right], \\ \varepsilon_1(\lambda) &:= \Pr \left[\begin{array}{l} \mathcal{D}_2(\mathbf{st}, \text{tr}) = 1 \wedge \\ \mathcal{R}_{p,n}(\mathbf{i}, \mathbf{x}, \mathbf{w}) \end{array} \left| \begin{array}{l} (\mathbf{srs}, \mathbf{td}_{\mathbf{srs}}) \leftarrow \text{KGen}(\mathbf{p}, n); (\mathbf{i}, \mathbf{x}, \mathbf{w}, \mathbf{st}) \leftarrow \mathcal{D}_1(\mathbf{srs}); \\ (\mathbf{ek}_i, \mathbf{vk}_i) \leftarrow \text{Derive}(\mathbf{srs}, \mathbf{i}); \\ \text{tr} \leftarrow \langle \text{Sim}(\mathbf{srs}, \mathbf{td}_{\mathbf{srs}}, \mathbf{i}, \mathbf{x}), \mathbf{V}(\mathbf{vk}_i, \mathbf{x}) \rangle \end{array} \right. \right]. \end{aligned}$$

We say that Π has statistical honest verifier zero-knowledge when $\varepsilon(\lambda)$ is negligible and perfect zero-knowledge when $\varepsilon(\lambda) = 0$.

Succinctness. Π is *succinct* if the running time of $\mathbf{V}$ is $\text{poly}(\lambda + |\mathbf{x}| + \log |\mathbf{w}|)$ and the communication size is $\text{poly}(\lambda + \log |\mathbf{w}|)$.

Π is *updatable* [GKM⁺18], if the SRS can be sequentially updated by many updaters, such that knowledge-soundness holds if either the original SRS creator or one of the updaters is honest.

When we have a public-coin protocol with a constant number of rounds, we can apply the Fiat-Shamir heuristic [FS87] to obtain a zk-SNARK.

3 ARSDH: Underlying Security Assumption

For a set $\mathcal{S}$, $\mathbf{Z}_{\mathcal{S}}(X) := \prod_{s \in \mathcal{S}} (X - s)$ is its vanishing polynomial. We need a new assumption, ARSDH, an adaptive version of the following known assumption.

⁶ Pavel Hubáček pointed out that we consider expected polynomial time respect to sampled $(\mathbf{p}, \mathbf{srs}, \mathbf{x})$. In a standard definition of witness-extended emulation [Lin01], the emulator needs to be expected polynomial time concerning *any* input.

Definition 3 (RSDH [GR19]). Let $n \in \text{poly}(\lambda)$ and Pgen be a bilinear-group generator. Let $\mathcal{S} = \{\alpha_j\} \subset \mathbb{F}$ be any set of size $n + 1$. Then, the $\mathcal{S}$ -RSDH (Rational Strong Diffie-Hellman) assumption holds for Pgen in $\mathbb{G}_1$, if for any PPT $\mathcal{A}$, the following probability is negligible: $\text{Adv}_{\text{Pgen}, 1, n, \mathcal{S}, \mathcal{A}}^{\text{rsdh}}(\lambda) :=$

$$\Pr \left[\begin{array}{l} [g]_1 \neq [0]_1 \wedge \\ [g]_1 \bullet [1]_2 = [\varphi]_1 \bullet [\mathbf{Z}_{\mathcal{S}}(\sigma)]_2 \end{array} \middle| \begin{array}{l} \mathbf{p} \leftarrow \text{Pgen}(1^\lambda); \sigma \leftarrow_{\$} \mathbb{Z}_p; \\ \text{ck} \leftarrow ([(\sigma^i)_{i=0}^n]_1, [(\sigma^i)_{i=0}^{n+1}]_2); \\ [g, \varphi]_1 \leftarrow \mathcal{A}(\text{ck}, \mathcal{S}) \end{array} \right].$$

(The condition $[g]_1 \bullet [1]_2 = [\varphi]_1 \bullet [\mathbf{Z}_{\mathcal{S}}(\sigma)]_2$ is equivalent to $\frac{1}{\mathbf{Z}_{\mathcal{S}}(\sigma)} \cdot [g]_1 = [\varphi]_1$.)

Definition 4 (New Assumption ARSDH). The adaptive $(n + 1)$ -RSDH assumption holds for Pgen in $\mathbb{G}_1$ if $\mathcal{S}$ -RSDH holds for Pgen in $\mathbb{G}_1$ even when the adversary can choose itself a set $\mathcal{S}$ of size $n + 1$. That is, if for any PPT $\mathcal{A}$, the following probability is negligible: $\text{Adv}_{\text{Pgen}, 1, n, \mathcal{A}}^{\text{arsdh}}(\lambda) :=$

$$\Pr \left[\begin{array}{l} \mathcal{S} \subset \mathbb{F} \wedge |\mathcal{S}| = n + 1 \wedge [g]_1 \neq [0]_1 \wedge \\ [g]_1 \bullet [1]_2 = [\varphi]_1 \bullet [\mathbf{Z}_{\mathcal{S}}(\sigma)]_2 \end{array} \middle| \begin{array}{l} \mathbf{p} \leftarrow \text{Pgen}(1^\lambda); \sigma \leftarrow_{\$} \mathbb{Z}_p; \\ \text{ck} \leftarrow ([(\sigma^i)_{i=0}^n]_1, [1, \sigma]_2); \\ (\mathcal{S}, [g, \varphi]_1) \leftarrow \mathcal{A}(\text{ck}) \end{array} \right].$$

The $(\mathcal{S})$ -RSDH assumption from [GR19] is stronger than ARSDH in the sense that it gives the adversary $[(\sigma^i)_{i=0}^n]_1, [(\sigma^i)_{i=0}^{n+1}]_2$ as an input. The additional input elements were needed for the assumption to be publicly verifiable, meaning breaking the success of the adversary can be tested only by knowing ck (knowing σ itself is unnecessary). In our application, public verification is not important. On the other hand, [GR19] assumed $\mathcal{S}$ -RSDH (for a specific set $\mathcal{S}$) while ARSDH must be secure against an adversary that adaptively chooses $\mathcal{S}$.

In our constructions, we need both ARSDH and SDH. The latter is needed for the evaluation binding of the KZG commitment. To simplify the assumption zoo, we prove the following lemma. Note that the adaptive choice of $\mathcal{S}$ in ARSDH is needed for the reduction to work (in addition to being needed in Section 4.1).

Lemma 1. $(n + 1)$ -ARSDH implies n -SDH.

Proof. Let $\mathcal{A}$ be an n -SDH adversary. We construct the following $(n + 1)$ -ARSDH adversary $\mathcal{B}(\text{ck})$:

1. Obtain $(c, [\varphi]_1 = \frac{1}{\sigma+c}[1]_1) \leftarrow \mathcal{A}(\text{ck})$.
2. Abort if $\mathcal{A}(\text{ck})$ did not succeed, i.e. if $[\varphi]_1 \bullet [\sigma + c]_2 \neq [1]_T$ or $\sigma + c = 0$.
3. Choose *any* set $\mathcal{S}$ of size $n + 1$ that contains $-c$, but not σ .
4. Set $g(X) \leftarrow \mathbf{Z}_{\mathcal{S}}(X)/(X + c) \in \mathbb{F}[X]$, with degree n .
5. Return $(\mathcal{S}, [g(\sigma), \varphi]_1)$.

Clearly, $\mathcal{B}$ has broken RSDH since $\frac{1}{\mathbf{Z}_{\mathcal{S}}(\sigma)}[g(\sigma)]_1 = [\varphi]_1$ and $g(\sigma) \neq 0$. $\square$

In particular, this means that ARSDH implies that KZG is evaluation binding. In Section B, we prove the security of ARSDH in the algebraic group model with oblivious sampling (AGMOS) [LPS23]. This is the recent variant of AGM [FKL18], which additionally allows oblivious sampling of group elements. It does not mean that the security of our protocols relies on AGM/AGMOS. Instead, we use AGMOS only as an additional sanity check.

4 Special Soundness of KZG

In the following sections, we define two security notions for non-interactive polynomial commitment schemes: special soundness and black-box extractability (BBE). We prove that KZG satisfies both notions under ARSDH.

We model a non-interactive polynomial commitment scheme (e.g., KZG) as a three-message protocol, where the first message is a commitment, the second is the verifier's query (evaluation point) α , and the third is an evaluation η together with an evaluation proof π .

Notation. We call $\mathbf{tr} := (C, \alpha, \eta, \pi)$ a *transcript*. For a fixed $\mathbf{ck}$, $\mathbf{tr}$ is *accepting* if $V(\mathbf{ck}, \mathbf{tr}) = 1$. Let $n \geq 1$ be an integer. We say that an $(n+1)$ -tuple $\mathbf{tr} = \{\mathbf{tr}_j = (C_j, \alpha_j, \eta_j, \pi_j) : j \in [0, n]\}$ is *admissible*, if (1) $C_i = C_j =: C$ for all $i, j \in [0, n]$, and (2) $\alpha_i \neq \alpha_j$ for $i \neq j$. We say $\mathbf{tr}$ is *accepting* if each $\mathbf{tr}_j$ is accepting.

For a non-interactive PCS $\mathbf{PC}$, $\lambda \in \mathbb{N}$, $n \in \text{poly}(\lambda)$, $\mathbf{p} \in \text{Pgen}(1^\lambda)$, $\mathbf{ck} \in \mathbf{PC.KGen}(\mathbf{p}, n)$ (that encodes implicitly information about $\mathbf{PC}$, λ , n , and $\mathbf{p}$), and an admissible $(n+1)$ -tuple $\mathbf{tr}$, we define the following two relations:

$$\begin{aligned} \mathcal{R}_{\mathbf{ck}} &:= \{(C, f) : C = \mathbf{PC.Com}(\mathbf{ck}, f) \wedge \deg f \leq n\} , \\ \mathcal{R}_{\mathbf{ck}, \mathbf{tr}} &:= \{(C, f) : (C, f) \in \mathcal{R}_{\mathbf{ck}} \wedge \forall j \in [0, n]. f(\alpha_j) = \eta_j\} . \end{aligned} \quad (1)$$

That is, (C, f) belongs to $\mathcal{R}_{\mathbf{ck}}$ if $f(X)$ is a valid opening of C . Moreover, (C, f) belongs to $\mathcal{R}_{\mathbf{ck}, \mathbf{tr}}$ if it belongs to $\mathcal{R}_{\mathbf{ck}}$ and in addition, $f(\alpha_j) = \eta_j$ for all $j \in [0, n]$.

4.1 Special Soundness

Next, we define a variant of the standard special soundness [CDS94] notion for *non-interactive* polynomial commitment schemes.

Definition 5 (Special Soundness). Let $n \in \text{poly}(\lambda)$ with $n \geq 1$. A non-interactive polynomial commitment scheme $\mathbf{PC}$ is computationally $(n+1)$ -special-sound for Pgen , if there exists a PPT extractor Ext_{ss} , such that for any PPT adversary $\mathcal{A}_{\text{ss}}$, $\text{Adv}_{\text{Pgen}, \mathbf{PC}, \text{Ext}_{\text{ss}}, n+1, \mathcal{A}_{\text{ss}}}^{\text{ss}}(\lambda) :=$

$$\Pr \left[\begin{array}{l} \mathbf{tr} = (\mathbf{tr}_j)_{j=0}^n \wedge \\ \forall j \in [0, n]. \left(\begin{array}{l} \mathbf{tr}_j = (C, \alpha_j, \eta_j, \pi_j) \\ \wedge V(\mathbf{ck}, \mathbf{tr}_j) = 1 \end{array} \right) \\ \wedge (\forall i \neq j. \alpha_i \neq \alpha_j) \wedge (C, f) \notin \mathcal{R}_{\mathbf{ck}, \mathbf{tr}} \end{array} \middle| \begin{array}{l} \mathbf{p} \leftarrow \text{Pgen}(1^\lambda); \\ (\mathbf{ck}, \mathbf{tk}) \leftarrow \mathbf{KGen}(\mathbf{p}, n); \\ \mathbf{tr} \leftarrow \mathcal{A}_{\text{ss}}(\mathbf{ck}); \\ f \leftarrow \text{Ext}_{\text{ss}}(\mathbf{ck}, \mathbf{tr}) \end{array} \right] \approx_\lambda 0 .$$

Intuitively, this definition states that if $\mathcal{A}_{\text{ss}}$ produces an accepting admissible $(n+1)$ -tuple $\mathbf{tr}$, then one can extract a degree- $\leq n$ polynomial $f(X)$ that agrees with all the transcripts (i.e., with all $n+1$ polynomial openings).

Theorem 1. If the $(n+1)$ -ARSDH assumption holds, then KZG for degree $\leq n$ polynomials is computationally $(n+1)$ -special-sound: There exists a DPT extractor Ext_{ss} , such that for any PPT $\mathcal{A}_{\text{ss}}$, there exists a PPT $\mathcal{B}$, such that $\text{Adv}_{\text{Pgen}, \mathbf{PC}, \text{Ext}_{\text{ss}}, n+1, \mathcal{A}_{\text{ss}}}^{\text{ss}}(\lambda) \leq \text{Adv}_{\text{Pgen}, 1, n+1, \mathcal{B}}^{\text{arsdh}}(\lambda)$.

We fix some notation and state a technical lemma before proving Theorem 1. Let $\mathcal{I} := [0, n]$ and fix any set $\mathcal{S} := \{\alpha_j\}_{j \in \mathcal{I}}$ with $\alpha_i \neq \alpha_j$. As before, let $\mathbf{Z}_{\mathcal{S}}(X) := \prod_{j \in \mathcal{I}} (X - \alpha_j)$ be the vanishing polynomial of $\mathcal{S}$. For $j \in \mathcal{I}$, let

$$\ell_j^{\mathcal{I}}(X) := \prod_{i \neq j \in \mathcal{I}} \frac{X - \alpha_i}{\alpha_j - \alpha_i}$$

be the j th Lagrange polynomial of $\mathcal{S}$ over $\mathcal{I}$. Let

$$\mathbf{d}_j^{\mathcal{I}} := \frac{1}{\mathbf{Z}_{\mathcal{S} \setminus \{\alpha_j\}}(\alpha_j)} = \frac{1}{\prod_{i \neq j \in \mathcal{I}} (\alpha_j - \alpha_i)} .$$

Clearly,

$$\ell_j^{\mathcal{I}}(X) = \frac{\mathbf{Z}_{\mathcal{S}}(X) \mathbf{d}_j^{\mathcal{I}}}{X - \alpha_j} . \quad (2)$$

In Lemma 2, we generalize a batching technique of Tomescu et al. [TAB⁺20] from the case $\alpha_j \in \langle \omega \rangle$ to any $\alpha_j \in \mathbb{F}$.⁷

Lemma 2 (Batching lemma). *Let $\mathcal{S} = \{\alpha_j\}_{j \in \mathcal{I}}$ with $\alpha_i \neq \alpha_j$. Assume that for all $j \in \mathcal{I}$,*

$$[\mathbf{c} - \eta_j]_1 \bullet [1]_2 = [\varphi_j]_1 \bullet [\sigma - \alpha_j]_2 \quad (3)$$

for some $[\mathbf{c}]_1$, $[\varphi_j]_1$, and η_j . Then,

$$[\mathbf{c} - L(\sigma)]_1 \bullet [1]_2 = [\varphi]_1 \bullet [\mathbf{Z}_{\mathcal{S}}(\sigma)]_2 , \quad (4)$$

where $[\varphi]_1 := \sum_{j \in \mathcal{I}} \mathbf{d}_j^{\mathcal{I}} [\varphi_j]_1$ and $L(X) := \sum_{j \in \mathcal{I}} \eta_j \ell_j^{\mathcal{I}}(X)$ is the low-degree extension of $\{\eta_j\}$.

Proof. Let us first handle the case $\sigma = \alpha_{j_0}$ for some j_0 . In this case, Eq. (3) tells us that $\mathbf{c} = \eta_{j_0}$. Moreover, $\ell_j^{\mathcal{I}}(\sigma) = 0$ for $j \neq j_0$ while

$$\ell_{j_0}^{\mathcal{I}}(X) = \mathbf{d}_{j_0}^{\mathcal{I}} \prod_{i \neq j_0} (X - \alpha_i) = \frac{\mathbf{Z}_{\mathcal{S} \setminus \{\sigma\}}(X)}{\mathbf{Z}_{\mathcal{S} \setminus \{\sigma\}}(\sigma)} .$$

Since $\mathbf{d}_{j_0}^{\mathcal{I}} = 1/\mathbf{Z}_{\mathcal{S} \setminus \{\sigma\}}(\sigma)$, we get that $L(\sigma) = \sum_{j \in \mathcal{I}} \eta_j \ell_j^{\mathcal{I}}(\sigma) = \eta_{j_0} = \mathbf{c}$ and Eq. (4) holds.

From now on, assume $\sigma \notin \mathcal{S}$. Define implicitly $\varphi := (\mathbf{c} - L(\sigma))/\mathbf{Z}_{\mathcal{S}}(\sigma) = \mathbf{c}/\mathbf{Z}_{\mathcal{S}}(\sigma) - L(\sigma)/\mathbf{Z}_{\mathcal{S}}(\sigma)$. From Eq. (2), we get

$$\frac{L(\sigma)}{\mathbf{Z}_{\mathcal{S}}(\sigma)} = \frac{\sum_{j \in \mathcal{I}} \eta_j \ell_j^{\mathcal{I}}(\sigma)}{\mathbf{Z}_{\mathcal{S}}(\sigma)} = \frac{\sum_{j \in \mathcal{I}} \eta_j \mathbf{Z}_{\mathcal{S}}(\sigma) \mathbf{d}_j^{\mathcal{I}} / (\sigma - \alpha_j)}{\mathbf{Z}_{\mathcal{S}}(\sigma)} = \sum_{j \in \mathcal{I}} \frac{\mathbf{d}_j^{\mathcal{I}} \eta_j}{\sigma - \alpha_j} .$$

⁷ [TAB⁺20] used this technique of batching polynomial commitment openings to improve on efficiency, while we use it for the security proof. Without using Lemma 2, we have a somewhat uglier assumption, where $\mathcal{A}_{\text{ss}}$ returns $[\varphi_j]_1$ for every $j \in \mathcal{I}$, and $n + 1$ equalities $[g]_1 \bullet [1]_2 = [\varphi_j]_1 \bullet [\sigma - \alpha_j]_2$ hold individually.

$\text{Ext}_{\text{ss}}(\text{ck} = ([(\sigma^i)_{i=0}^n]_1, [1, \sigma]_2), \mathbf{tr})$	$\mathcal{B}(\text{ck} = ([(\sigma^i)_{i=0}^n]_1, [1, \sigma]_2))$
if $\exists i \neq j. \alpha_i = \alpha_j$ then return $\perp$; if $\exists j \in \mathcal{I} : \mathbf{V}(\text{ck}, \text{tr}_j) = 0$ then return $\perp$; $L(X) \leftarrow \sum_{j \in \mathcal{I}} \eta_j \ell_j^{\mathcal{I}}(X) \in \mathbb{F}_{\leq n}[X]$; $[g]_1 \leftarrow [c - L(\sigma)]_1$; if $[g]_1 = [0]_1$ then return $L(X)$; return $\perp$;	$\mathbf{tr} \leftarrow \mathcal{A}_{\text{ss}}(\text{ck}); \quad // \text{tr}_j = ([c]_1, \alpha_j, \eta_j, [\varphi_j]_1)$ if $\exists i \neq j. \alpha_i = \alpha_j$ then return $\perp$; if $\exists j \in \mathcal{I} : \mathbf{V}(\text{ck}, \text{tr}_j) = 0$ then return $\perp$; $L(X) \leftarrow \sum_{j \in \mathcal{I}} \eta_j \ell_j^{\mathcal{I}}(X) \in \mathbb{F}_{\leq n}[X]$; $[g]_1 \leftarrow [c - L(\sigma)]_1$; if $[g]_1 = [0]_1$ then return $\perp$; $[\varphi]_1 \leftarrow \sum_{j \in \mathcal{I}} \mathbf{d}_j^{\mathcal{I}}[\varphi_j]_1$; return $(\mathcal{S} \leftarrow \{\alpha_j\}, [g, \varphi]_1)$;

Fig. 1. The extractor Ext_{ss} and the ARSDH reduction $\mathcal{B}$ in the proof of Theorem 1

We also get from Eq. (2) that $\frac{1}{\mathbf{Z}_{\mathcal{S}}(X)} = \frac{1}{\mathbf{Z}_{\mathcal{S}}(X)} \sum_{j \in \mathcal{I}} \ell_j^{\mathcal{I}}(X) = \sum_{j \in \mathcal{I}} \frac{\mathbf{d}_j^{\mathcal{I}}}{X - \alpha_j}$. Thus, $c/\mathbf{Z}_{\mathcal{S}}(\sigma) = \sum_{j \in \mathcal{I}} \mathbf{d}_j^{\mathcal{I}} c/(\sigma - \alpha_j)$. Hence,

$$\varphi = \sum_{j \in \mathcal{I}} \frac{\mathbf{d}_j^{\mathcal{I}} c}{\sigma - \alpha_j} - \sum_{j \in \mathcal{I}} \frac{\mathbf{d}_j^{\mathcal{I}} \eta_j}{\sigma - \alpha_j} = \sum_{j \in \mathcal{I}} \mathbf{d}_j^{\mathcal{I}} \frac{c - \eta_j}{\sigma - \alpha_j} = \sum_{j \in \mathcal{I}} \mathbf{d}_j^{\mathcal{I}} \varphi_j.$$

The last equality holds since $\varphi_j = (c - \eta_j)/(\sigma - \alpha_j)$. This proves the lemma. $\square$

Proof (Of Theorem 1). Let $\mathcal{A}_{\text{ss}}$ be any PPT adversary in the computational special soundness game. We construct the following extractor Ext_{ss} . (See Fig. 1 for a formal description.) Let $n \in \text{poly}(\lambda)$, $\mathbf{p} \leftarrow \text{Pgen}(1^\lambda)$, and $(\text{ck}, \text{tk}) \leftarrow \text{KGen}(\mathbf{p}, n)$. Ext_{ss} obtains ck and $\mathbf{tr}$, where $\mathbf{tr}$ is an $(n+1)$ -tuple of transcripts $\text{tr}_j = ([c]_1, \alpha_j, \eta_j, [\varphi_j]_1)$. Define $\mathcal{S} := \{\alpha_j\}_{j \in \mathcal{I}}$.⁸ When $\mathcal{A}_{\text{ss}}$ produces a successful attack, α_j are pairwise different ($\mathbf{tr}$ is admissible), and the KZG verifier accepts each transcript. That is, for all $j \in \mathcal{I}$, Eq. (3) holds.

Ext_{ss} interpolates a polynomial $L(X)$ of degree $\leq n$ such that $L(\alpha_j) = \eta_j$ for every $j \in \mathcal{I}$. That is, $L(X) = \sum_{j \in \mathcal{I}} \eta_j \ell_j^{\mathcal{I}}(X)$. If $[c]_1 = [L(\sigma)]_1$, Ext_{ss} outputs $L(X)$. Observe that in this case, $\deg L(X) \leq n$ and $L(\alpha_j) = \eta_j$ for all $j \in \mathcal{I}$; thus, $(C, L) \in \mathcal{R}_{\text{ck}, \mathbf{tr}}$ as required in Definition 5. Otherwise, Ext_{ss} outputs $\perp$.

Let bad be the event that $c \neq L(\sigma)$ but the verifier accepts all transcripts in $\mathbf{tr}$. In Fig. 1, we depict a reduction $\mathcal{B}$ that breaks ARSDH whenever bad happens. $\mathcal{B}$ runs $\mathcal{A}_{\text{ss}}$ to obtain transcripts and then computes $L(X)$ just as the extractor. If $[c]_1 = [L(\sigma)]_1$, $\mathcal{B}$ outputs $\perp$ (in this case, the extractor succeeds). Otherwise, $\mathcal{B}$ proceeds and computes $[g]_1 \leftarrow [c - L(\sigma)]_1$ and $[\varphi]_1 \leftarrow \sum_{j \in \mathcal{I}} \mathbf{d}_j^{\mathcal{I}}[\varphi_j]_1$. Since Eq. (3) holds for all $j \in \mathcal{I}$, we get from Lemma 2 that Eq. (4) holds, that is, $[g]_1 \bullet [1]_2 = [\varphi]_1 \bullet [\mathbf{Z}_{\mathcal{S}}(\sigma)]_2$. When bad happens, $[g]_1 \neq [0]_1$ and thus $\mathcal{B}$ breaks the

⁸ Note that if $\mathcal{S}$ contains σ then $\mathcal{A}_{\text{ss}}$ has broken the $(n, 1)$ -PDL assumption, and thus also the ARSDH assumption. However, the following proof also goes through when $\mathcal{S}$ contains σ , and thus we do not have to consider the case $\sigma \in \mathcal{S}$ separately.

```

ExtrwP*(st,·)(ck, tr0)
Parse tr0 = (C, α0, η0, π0);
if V(ck, C, α0, η0, π0) = 0 then return ⊥; fi
j ← 1; T ← Fp \ {α0};
while j ≤ n ∧ T ≠ ∅ do
  α ←$ T; (η, π) ← P*(st, α);
  if V(ck, C, α, η, π) = 1 then trj ← (C, α, η, π); j ← j + 1; fi
  T ← T \ {α};
endwhile
if j < n then return ⊥; fi
return tr ← (tr1, ..., trn);

```

Fig. 2. Extractor Ext_{rw} from Theorem 2.

ARSDH assumption by returning $[g, \varphi]_1$. Thus, $\Pr[\mathcal{B} \text{ breaks ARSDH}] = \Pr[\text{bad}]$. Summarizing, $\text{Adv}_{\text{Pgen, PC, Ext}_{\text{ss}}, n+1, \mathcal{A}_{\text{ss}}}^{\text{ss}}(\lambda) \leq \text{Adv}_{\text{Pgen, 1, n+1, B}}^{\text{arsdh}}(\lambda)$. $\square$

5 Rewinding Lemma

Next, we prove a generic information-theoretical rewinding lemma. Intuitively, if the adversary can produce an accepting transcript for a random challenge α_0 , there exists an efficient extractor that can recover n more accepting transcripts for $n \in \text{poly}(\lambda)$. Note that ε_{ss} is a function of $\mathbf{p}$ since $|\mathcal{F}_{\mathbf{p}}|$ is a function of $\mathbf{p}$.

Theorem 2. Fix $\mathcal{F}_{\mathbf{p}}$ as a function of $\mathbf{p}$. For all DPT $\mathbf{P}^*$ and $n \in \text{poly}(\lambda)$, there exists an expected PT extractor Ext_{rw}^9 , such that for any unbounded $\mathcal{A}$, $\varepsilon_{\text{ss}}(\mathbf{p}) > 1 - n/|\mathcal{F}_{\mathbf{p}}|$, where for every $\mathbf{p} \in \text{Pgen}(1^\lambda)$, $\varepsilon_{\text{ss}}(\mathbf{p}) :=$

$$\Pr \left[\begin{array}{l} V(\text{ck}, \text{tr}_0) = 1 \Rightarrow \\ \forall j \in [1, n]. V(\text{ck}, \text{tr}_j) = 1 \end{array} \middle| \begin{array}{l} (\text{ck}, \text{tk}) \leftarrow \text{KGen}(\mathbf{p}, n); (C, \text{st}) \leftarrow \mathcal{A}(\text{ck}); \\ \alpha_0 \leftarrow_{\$} \mathcal{F}_{\mathbf{p}}; (\eta_0, \pi_0) \leftarrow \mathbf{P}^*(\text{st}, \alpha_0); \\ \text{tr}_0 \leftarrow (C, \alpha_0, \eta_0, \pi_0); \\ \mathbf{tr} \leftarrow \text{Ext}_{\text{rw}}^{\mathbf{P}^*(\text{st}, \cdot)}(\text{ck}, \text{tr}_0) \end{array} \right],$$

and $\mathbf{tr} = (\text{tr}_1, \dots, \text{tr}_n)$ consists of transcripts $\text{tr}_j = (C, \alpha_j, \eta_j, \pi_j)$ for $j \in [1, n]$, where C is the same as in tr_0 and $\alpha_j \in \mathcal{F}_{\mathbf{p}}$ are pairwise distinct. In particular, Ext_{rw} makes an expected number of n queries to $\mathbf{P}^*$.

Proof. Our proof strategy roughly follows [ACK21]. In Fig. 2, we depict the extractor Ext_{rw} . Intuitively, Ext_{rw} runs $\mathbf{P}^*(\text{st}, \cdot)$ on distinct uniformly random challenges $\alpha \in \mathcal{F}_{\mathbf{p}}$ until it either finds n additional accepting transcripts or the whole challenge set $\mathcal{F}_{\mathbf{p}}$ is exhausted.

Let $r_{\mathbf{p}}$, r_{ck} , and $r_{\mathcal{A}}$ be the randomizers used in creating $\mathbf{p} \leftarrow \text{Pgen}(1^\lambda; r_{\mathbf{p}})$, $\text{ck} \leftarrow \text{KGen}(\mathbf{p}, n; r_{\text{ck}})$, and $(C, \text{st}) \leftarrow \mathcal{A}(\text{ck}; r_{\mathcal{A}})$. Consider a Boolean matrix H

⁹ We consider expected polynomial time with respect to a sampled input (ck, tr_0) . See also the remark in Section 2.2.

where the rows are indexed by the set $\text{Rows} := \{\bar{r} = (r_p, r_{ck}, r_A) : r_p, r_{ck}, r_A \in \{0, 1\}^{\text{poly}(\lambda)}\}$ and the columns are indexed by the verifier challenges from $\mathcal{F}_p$. We implicitly set $H_{\bar{r}, \alpha} = 1$ iff $V(\text{ck}, C, \alpha, \eta, \pi) = 1$, where the parameters $(p, \text{ck}, C, \text{st})$ are created using the coins $\bar{r}$ and $(\eta, \pi) \leftarrow P^*(\text{st}, \alpha)$.

Probability analysis. Consider the game described in the lemma's statement. W.r.t. this game, let A be the event that $V(\text{ck}, \text{tr}_0) = 1$ and let B be the event that $\forall j \in [1, n], V(\text{ck}, \text{tr}_j) = 1$. Then,

$$\begin{aligned} \varepsilon_{\text{ss}}(p) &= \Pr[A \Rightarrow B] = \Pr[A \wedge (A \Rightarrow B)] + \Pr[\neg A \wedge (A \Rightarrow B)] \\ &= \Pr[A \wedge B] + \Pr[\neg A] . \end{aligned}$$

Let $R := |\text{Rows}|$. For $j \leq |\mathcal{F}_p|$, let R_j be the number of rows in H with exactly j ones. Thus, $\Pr[A] = (\sum_{j=1}^{|\mathcal{F}_p|} j R_j) / (R \cdot |\mathcal{F}_p|)$ is the fraction of ones in H . The event $A \wedge B$ happens iff tr_0 is accepting and it comes from a row $\bar{r} \in \text{Rows}$ with at least $n + 1$ ones. Thus,

$$\Pr[A \wedge B] = \frac{\sum_{j=n+1}^{|\mathcal{F}_p|} j R_j}{R \cdot |\mathcal{F}_p|} = \frac{\sum_{j=1}^{|\mathcal{F}_p|} j R_j}{R \cdot |\mathcal{F}_p|} - \frac{\sum_{j=1}^n j R_j}{R \cdot |\mathcal{F}_p|} = \Pr[A] - \frac{\sum_{j=1}^n j R_j}{R \cdot |\mathcal{F}_p|} .$$

Next, $\sum_{j=1}^n j R_j$ is largest when all R rows have exactly n ones, i.e., $R_n = R$ and $R_j = 0$ for $j < n$. Thus, $(\sum_{j=1}^n j R_j) / (R \cdot |\mathcal{F}_p|) \leq nR / (R \cdot |\mathcal{F}_p|) = n / |\mathcal{F}_p|$ and $\Pr[A \wedge B] \geq \Pr[A] - n / |\mathcal{F}_p|$. Finally, $\varepsilon_{\text{ss}}(p) \geq (\Pr[A] - n / |\mathcal{F}_p|) + \Pr[\neg A] = 1 - n / |\mathcal{F}_p|$.

Expected number of queries. Let Q denote the number of queries Ext_{rw} makes to P^* . Since the total running time of Ext_{rw} is $\text{poly}(\lambda) \cdot Q$, it is sufficient to only analyze Q . Consider the case that $H_{\bar{r}, \alpha_0} = 1$ (therefore, A happened); then, the extractor in Fig. 2 will not abort on the second step but enters the **while** loop. The **while** loop in the extractor can be viewed as sampling without replacement from a finite binary-classified population.

Recall the negative hypergeometric distribution (NHG) is the distribution of X in the next game: given a bin with N balls of which K are marked, X is the number of sampled balls from the bin (without replacements) until we get $k \leq K$ marked balls. The expected value of X is $\mathbb{E}[\text{NHG}_{N, K, k}] = k(N+1)/(K+1)$. The number of iterations of the **while** loop corresponds to an NHG random variable with the following parameters, where $\delta_{\bar{r}}$ is the fraction of ones in $H_{\bar{r}}$ ($\bar{r}$ th row).

- $N = |\mathcal{F}_p| - 1$ (the number of possible challenges except α_0),
- $K = \delta_{\bar{r}} |\mathcal{F}_p| - 1$ (the number of ones in $H_{\bar{r}}$, except the entry $H_{\bar{r}, \alpha_0}$),
- $k = n$ (the additional number of accepting transcripts Ext_{rw} needs to find).

If there are at least $n + 1$ entries in the row $\bar{r}$, the expected number of iterations in the **while** loop to get n ones is $k(N+1)/(K+1) = n|\mathcal{F}_p|/(\delta_{\bar{r}}|\mathcal{F}_p|) = n/\delta_{\bar{r}}$. On the other hand, if there are less than $n + 1$ ones in the row $\bar{r}$, then the **while** loop checks all the $|\mathcal{F}_p| - 1$ entries in the row. In this case, $K = \delta_{\bar{r}}|\mathcal{F}_p| - 1 \leq n - 1$ and thus $|\mathcal{F}_p| \leq n/\delta_{\bar{r}}$. Thus, the expected number of iterations in the **while** loop satisfies $|\mathcal{F}_p| - 1 < n/\delta_{\bar{r}}$. Hence, $\mathbb{E}[Q \mid A \wedge \bar{r}] \leq n/\delta_{\bar{r}}$.

The conditional probability, given the row $\bar{r}$, that the extractor does not abort before entering the **while** loop is $\delta_{\bar{r}} = \Pr[A \mid \bar{r}]$. Thus, conditioned on the row $\bar{r}$, the expected number of calls that the extractor makes is $\mathbb{E}[Q \mid \bar{r}] \leq \delta_{\bar{r}} \cdot n / \delta_{\bar{r}} = n$. The expected number of calls of Ext_{rw} can be computed as the average expected number of calls over the choice of $\bar{r}$:

$$\mathbb{E}[Q] = \sum_{\bar{r} \in \text{Rows}} \mathbb{E}[Q \mid \bar{r}] \Pr[\bar{r}] \leq \sum_{\bar{r}=1}^{|\text{Rows}|} \frac{n}{|\text{Rows}|} = n .$$

The claim follows. $\square$

6 Black-Box Extractability

Definition. For a single transcript tr , we write that $(C, f) \in \mathcal{R}_{\text{ck}, \text{tr}}$ iff $(C, f) \in \mathcal{R}_{\text{ck}} \wedge f(\alpha) = \eta$. (This is a particular case of Eq. (1) for $n = 0$.)

Definition 6 (Black-Box Extractability). A non-interactive polynomial commitment scheme PC is black-box extractable (BBE) for Pgen , if there exists an expected PPT black-box extractor Ext_{bb} , such that for all PPT $\mathcal{A}$, DPT P^* , $n \in \text{poly}(\lambda)$, and $\mathcal{F}_{\text{p}} \subseteq \mathbb{F}$ of size $|\mathcal{F}_{\text{p}}| = \lambda^{\omega(1)}$, $\text{Adv}_{\text{Pgen}, \text{PC}, \text{Ext}, n, \mathcal{A}, \text{P}^*}^{\text{bbe}}(\lambda) =$

$$\Pr \left[\begin{array}{l} V(\text{ck}, \text{tr}_0) = 1 \wedge \\ (C, f) \notin \mathcal{R}_{\text{ck}, \text{tr}_0} \end{array} \middle| \begin{array}{l} \text{p} \leftarrow \text{Pgen}(1^\lambda); (\text{ck}, \text{tk}) \leftarrow \text{KGen}(\text{p}, n); (C, \text{st}) \leftarrow \mathcal{A}(\text{ck}); \\ \alpha_0 \leftarrow \mathcal{F}_{\text{p}}; (\eta_0, \pi_0) \leftarrow \text{P}^*(\text{st}, \alpha_0); \text{tr}_0 \leftarrow (C, \alpha_0, \eta_0, \pi_0); \\ f \leftarrow \text{Ext}_{\text{bb}}^{\text{P}^*(\text{st}, \cdot)}(\text{ck}, \text{tr}_0); \end{array} \right] \approx_{\lambda} 0 .$$

Ext_{bb} can invoke $\text{P}^*(\text{st}, \cdot)$ with any challenge $\tilde{\alpha} \in \mathcal{F}_{\text{p}}$ to which $\text{P}^*(\text{st}, \tilde{\alpha})$ returns some tuple $(\tilde{\eta}, \tilde{\pi})$.

Note that st can contain information about ck and C . In Definition 6, P^* being deterministic means that we can rewind and restart P^* many times on the same state st , but different challenges.

We balanced Definition 6 so that it is weak enough for KZG to satisfy it and strong enough so that the SNARK compiler in Section 7 can use it. A crucial difference between Definition 1 and Definition 6 is that the evaluation point is sampled randomly from a large set in the latter. We need this property to prove Theorem 2. This difference with Definition 1 is why we must open each polynomial at a random evaluation point in our compiler in Section 7.

In Section C, we show that if the PCS is binding and BBE, it also has non-adaptive evaluation binding. This is a slightly weaker form of evaluation binding than usual, where the game picks the evaluation point randomly.

Security Reduction. We prove that if a non-interactive polynomial commitment scheme is computationally special-sound, it is also black-box extractable, as in Definition 6. The proof of Theorem 3 per se is not complicated if one assumes the results of Sections 4 and 5.

$\text{Ext}_{\text{bb}}^{\text{P}^*(\text{st}, \cdot)}(\text{ck}, \text{tr}_0)$	$\mathcal{A}_{\text{ss}}(\text{ck})$
1 : if $V(\text{ck}, \text{tr}_0) = 0$ then return $\perp$; fi	$(C, \text{st}) \leftarrow \mathcal{A}(\text{ck}); \alpha_0 \leftarrow \mathcal{F}_{\text{p}};$ $(\eta_0, \pi_0) \leftarrow \text{P}^*(\text{st}, \alpha_0);$ $\text{tr}_0 \leftarrow (C, \alpha_0, \eta_0, \pi_0);$ if $V(\text{ck}, \text{tr}_0) = 0$ then return $\perp$; fi
2 : $\text{tr} \leftarrow \text{Ext}_{\text{rw}}^{\text{P}^*(\text{st}, \cdot)}(\text{ck}, \text{tr}_0);$	$\text{tr} \leftarrow \text{Ext}_{\text{rw}}^{\text{P}^*(\text{st}, \cdot)}(\text{ck}, \text{tr}_0);$
3 : if $\exists j \in [1, n]. V(\text{ck}, \text{tr}_j) = 0$ then return $\perp$; fi $f \leftarrow \text{Ext}_{\text{ss}}(\text{ck}, \text{tr}_0 \parallel \text{tr});$	if $\exists j \in [1, n]. V(\text{ck}, \text{tr}_j) = 0$ then return $\perp$; fi
4 : return f ;	return $(\text{tr}_0 \parallel \text{tr});$

Fig. 3. The black-box extractor Ext_{bb} and the special-soundness adversary $\mathcal{A}_{\text{ss}}^{\text{P}^*(\text{st}, \cdot)}$ from Theorem 3.

Theorem 3. *If a non-interactive PCS PC is computational $(n + 1)$ -special-sound, it is black-box extractable.*

Proof. Assume PC is special-sound (see Definition 5) and let Ext_{ss} be the guaranteed PPT special soundness extractor. Let Ext_{rw} be the rewinding extractor from Fig. 2. Let $\mathcal{A}$ and P^* be the adversaries in the definition of black-box extraction.

Fig. 3 depicts the new black-box extractor Ext_{bb} . If the verifier does not accept tr_0 , there is no need to extract, and Ext_{bb} outputs $\perp$. Otherwise, Ext_{bb} calls Ext_{rw} to obtain n more accepting transcripts tr , resulting in an $(n + 1)$ -tuple $\text{tr}_0 \parallel \text{tr}$ of admissible transcripts. Ext_{bb} runs Ext_{ss} on $\text{tr}_0 \parallel \text{tr}$, obtaining a polynomial f . If both Ext_{rw} and Ext_{ss} succeed, then Ext_{bb} outputs f . Since Ext_{rw} runs in the expected PPT and Ext_{ss} in strict PPT, Ext_{bb} runs in expected PPT.

We next bound the probability that when running Ext_{bb} , $V(\text{ck}, \text{tr}_0) = 1$ but $(C, f) \notin \mathcal{R}_{\text{ck}, \text{tr}_0}$. Looking at Fig. 3, we can see that there are two different ways for this to happen.

- (i) Ext_{bb} returns $\perp$ in step 3. This corresponds to the event $\mathbf{F}_{\text{rw}}$ (“failure in rewinding”) that $V(\text{ck}, \text{tr}_0) = 1$ but $V(\text{ck}, \text{tr}_j) = 0$ for some $j \in [1, n]$.
- (ii) Ext_{bb} reaches step 4 and returns f , such that $(C, f) \notin \mathcal{R}_{\text{ck}, \text{tr}_0}$. This corresponds to the event $\mathbf{F}_{\text{ss}}$ (“failure in special soundness”) that $\forall j \in [0, n]. V(\text{ck}, \text{tr}_j) = 1$ but $(C, f) \notin \mathcal{R}_{\text{ck}, \text{tr}_0}$.

We next bound the probabilities that $\mathbf{F}_{\text{rw}}$ or $\mathbf{F}_{\text{ss}}$ happen.

Event $\mathbf{F}_{\text{rw}}$. By Theorem 2, $\mathbf{F}_{\text{rw}}$ happens with probability at most $n/|\mathcal{F}_{\text{p}}|$.

Event $\mathbf{F}_{\text{ss}}$. We bound the probability that $\mathbf{F}_{\text{ss}}$ happens by relying on special soundness (Definition 5). In Fig. 3, we depict a special soundness adversary $\mathcal{A}_{\text{ss}}$ that runs $\mathcal{A}$ and P^* internally. $\mathcal{A}_{\text{ss}}$ first runs $\mathcal{A}$ and P^* to create tr_0 as in Definition 6. Crucially, $\mathcal{A}_{\text{ss}}$ creates tr_0 from the correct distribution for Ext_{bb} . Then, $\mathcal{A}_{\text{ss}}$ executes Ext_{bb} up to step 3. Finally, $\mathcal{A}_{\text{ss}}$ outputs tr_0 together with n transcripts computed by Ext_{rw} . (However, it does not run Ext_{ss} .)

According to the special soundness definition, the event that the extractor outputs $f(X)$ such that $(C, f) \notin \mathcal{R}_{\text{ck}, \text{tr}_0}$, but $\forall j \in [0, n]. V(\text{ck}, \text{tr}_j) = 1$ is bounded $\text{Adv}_{\text{Pgen}, \text{PC}, \text{Ext}_{\text{ss}}, n+1, \mathcal{A}_{\text{ss}}}^{\text{ss}}(\lambda)$. This corresponds precisely to the event $\mathbf{F}_{\text{ss}}$. Thus, $\Pr[\mathbf{F}_{\text{ss}}] \leq \text{Adv}_{\text{Pgen}, \text{PC}, \text{Ext}_{\text{ss}}, n+1, \mathcal{A}_{\text{ss}}}^{\text{ss}}(\lambda)$. However, $\mathcal{A}_{\text{ss}}$ is an expected PPT adversary, but special soundness is defined only for strict PPT adversary. We observe that special soundness is a falsifiable assumption (it can be written as an interaction between a PPT challenger and an adversary, Section A). According to Lemma 6, since $\text{Adv}_{\text{Pgen}, \text{PC}, \text{Ext}_{\text{ss}}, n+1, \mathcal{A}'}^{\text{ss}}(\lambda)$ is negligible for any PPT $\mathcal{A}'$, then also $\text{Adv}_{\text{Pgen}, \text{PC}, \text{Ext}_{\text{ss}}, n+1, \mathcal{A}_{\text{ss}}}^{\text{ss}}(\lambda)$ is a negligible function.

Summing up the above results, we get the claim of the theorem. $\square$

Corollary 1. *If $(n+1)$ -ARSDH holds, then KZG for degree $\leq n$ polynomials is black-box extractable.*

Proof. Follows directly from Theorems 1 and 3. $\square$

7 Application to SNARKs

7.1 Polynomial IOP

The following definition of polynomial IOP (PIOP) roughly follows [SZ20].

Definition 7 (Polynomial IOP with Preprocessing). *Let $\mathcal{R}$ be an indexed relation, $\mathbb{F}$ a finite field, and $d \in \mathbb{N}$ a degree bound. A polynomial IOP for $\mathcal{R}$ with degree bound d is a tuple of PPT algorithms $\Pi_{\text{IOP}} = (\text{IOP.I}, \text{IOP.P}, \text{IOP.V})$, satisfying the following. 1. IOP.I takes $\mathfrak{i}$ for input and outputs a list of polynomials of degree at most d . 2. IOP.V gets oracle access to these polynomials. 3. $(\text{IOP.P}, \text{IOP.V})$ run a Rnds -round interactive protocol. 4. In each round, IOP.P sends polynomials $f_i(X) \in \mathbb{F}[X]$ of degree at most d to IOP.V . 5. IOP.V is an oracle machine with access to a list of oracles, containing one oracle for each polynomial it has received from the prover (and indexer). 6. At the end of each round, IOP.V can query oracles associated with $f_i(X)$ on a point $\alpha_j \in \mathbb{F}$; the oracle responds with the value $f_i(\alpha_j)$. 7. In the subsequent round, IOP.V sends oracle queries and challenges $z_k \in \mathbb{F}$ to IOP.P . 8. IOP.V is public coin.*

On common input $\mathfrak{x}$ and prover's witness $\mathfrak{w}$, the protocol transcript is denoted by the random variable $\langle \text{IOP.P}(\mathfrak{x}, \mathfrak{w}), \text{IOP.V}(\mathfrak{x}) \rangle = \text{tr}$. When the verifier accepts the transcript, we conveniently write $\text{tr} = 1$. $\text{IOP.V}^{(\mathfrak{i})}$ denotes that the verifier has oracle access to the polynomials outputted by the indexer.

A polynomial IOP must satisfy the following properties.

Perfect Completeness: for all $(\mathfrak{i}, \mathfrak{x}, \mathfrak{w}) \in \mathcal{R}$,

$$\Pr \left[\left\langle \text{IOP.P}(\mathfrak{x}, \mathfrak{w}), \text{IOP.V}^{(\mathfrak{i})}(\mathfrak{x}) \right\rangle = 1 \right] = 1.$$

Knowledge Soundness: There exists a PPT algorithm Ext^{10} , such that for any interactive algorithm $\mathcal{A}$, $\mathfrak{i} \in \mathbb{I}(\mathcal{R})$, and input $\mathfrak{x} \in \{0, 1\}^{\text{poly}(\lambda)}$ (not necessarily in

¹⁰ Note that [BFS20] allowed the extractor to rewind. We are unaware of any concrete polynomial IOP, which would need that. Thus, for the sake of simplicity we use a straight-line extractor.

the language), the following holds: $\text{Adv}_{\Pi, \text{Ext}, \mathcal{A}}^{\text{ks}}(\lambda) :=$

$$\Pr \left[\text{tr} = 1 \wedge (\mathbf{i}, \mathbb{x}, \mathbb{w}) \notin \mathcal{R} \mid \text{tr} \leftarrow \left\langle \mathcal{A}(\mathbf{i}, \mathbb{x}), \text{IOP.V}^{I(\mathbf{i})}(\mathbb{x}) \right\rangle; \mathbb{w} \leftarrow \text{Ext}(\mathbf{i}, \mathbb{x}, \text{tr}) \right] \approx_{\lambda} 0 .$$

7.2 Compiling Polynomial IOPs Into Arguments

We present a compiler that compiles a PIOP and a non-interactive polynomial commitment scheme (as defined in Section 2.1) into an interactive argument.

The SRS is the public key of the commitment scheme, and in the preprocessing phase, the commitments of the indexer’s polynomials are published. The interactive part of the argument runs PIOP, except with the following two changes:

- If the PIOP prover sends some polynomial f to the PIOP verifier, then $\mathbf{P}$ and $\mathbf{V}$ execute the following three-message subprotocol: the prover sends a commitment of f to the verifier, the verifier samples a new extraction point¹¹ $\chi \leftarrow_{\$} \mathcal{F}$ (here $|\mathcal{F}| = \lambda^{\omega(1)}$), and the prover opens the commitment of f at this point. The verifier checks that the opening proof verifies. We can think of this protocol as an interactive commitment phase of a new polynomial commitment scheme KZG^+ .
- If the PIOP verifier queries a previously sent polynomial f at some point α , the verifier sends α to the prover and the prover opens the commitment to f at the evaluation point α . The verifier checks that the opening proof verifies.

The rest of the PIOP is executed truthfully. In particular, the verifier checks that the PIOP verifier accepts the corresponding PIOP transcript that can be easily compiled from the verifier’s responses and evaluations sent by the prover.

It is important to observe that even if the original PIOP has zero knowledge, the compiled argument might not have. We make new queries (with extraction points) to the polynomials, which leak additional information. However, in efficient zk-SNARKs like Plonk, this can be tackled by adding extra randomness to polynomials to account for one more opening point.

For simplicity, we assume that the PIOP prover sends a single polynomial in each round. This assumption is without loss of generality: We can always write an equivalent PIOP, where instead of sending many polynomials per round, the prover sends each in a separate round, and the verifier’s intermediate responses are random challenges never used by the prover. In the compiled argument, these challenges can be removed, meaning there is no increase in the number of rounds.

Compiler Description. We introduce more notation to make it easier to explain rewinding. Let $(\text{IOP.I}, \text{IOP.P}, \text{IOP.V})$ be a polynomial IOP. We denote by $\text{st}_{r-1}^{\mathbf{V}}$ the verifier’s state at the beginning of round r . The verifier’s initial state is $\text{st}_0^{\mathbf{V}} = \mathbb{x}$. On input $(\text{st}_{r-1}^{\mathbf{V}}, \text{coins}_r)$, $\mathbf{V}$ outputs query points α_r , other challenge values $\mathbf{z}_r$, and labels¹² $\mathbf{g}_r$ for the oracles that need to be evaluated. More precisely, $\mathbf{g}_r$ is a vector of length $\ell_r = |\mathbf{g}_r| = |\alpha_r|$ containing labels for $\mathbf{V}$ ’s oracles

¹¹ In practice, $\mathcal{F} = \mathbb{F}$. After using Fiat-Shamir, the extraction point is a hash.

¹² They have to be labels, not polynomials, since technically $\mathbf{V}$ does not know the polynomials themselves. For simplicity, we will ignore this difference in the notation.

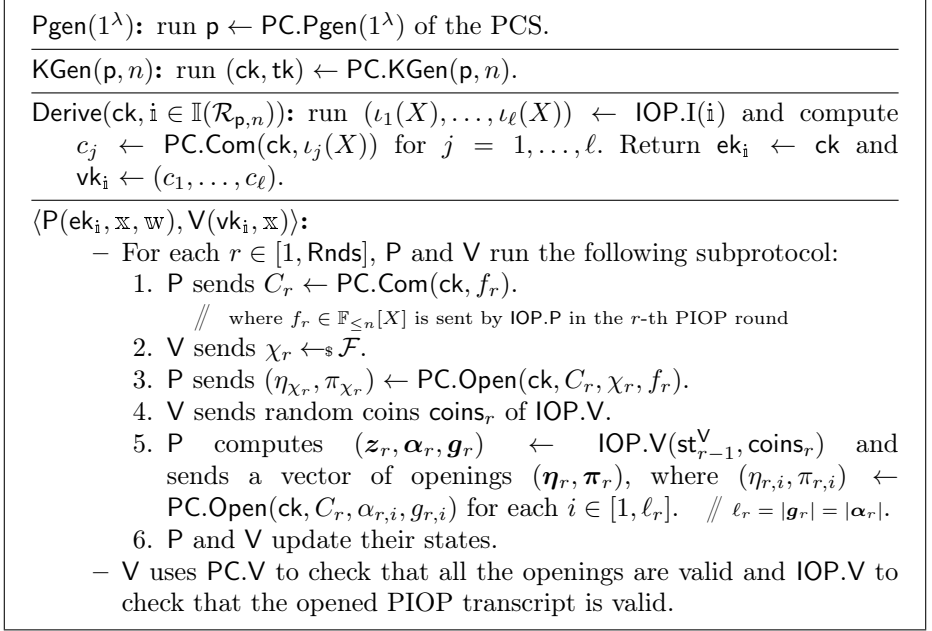


Fig. 4. Public-coin interactive argument system $\Pi = (\text{Pgen}, \text{KGen}, \text{Derive}, \text{P}, \text{V}, \text{Sim})$ for the relation $\mathcal{R}_{\mathbf{p}, n}$.

that need to be queried. An oracle can be from the prover or from the indexer (the same oracle may appear even multiple times in $\mathbf{g}_r$). The verifier queries $\eta_{r,i} \leftarrow g_{r,i}(\alpha_{r,i})$ for all $i \in [1, \ell_r]$, where $g_{r,i}$ is a label for one of V's oracles.

We depict the compiler, with full details, in Fig. 4. Here, $\Pi_{\text{IOP}} = (\text{IOP.I}, \text{IOP.P}, \text{IOP.V})$ is a Rnds -round PIOP for $\mathcal{R}_{\mathbf{p}, n}$ and PC is a non-interactive polynomial commitment scheme.

Security. We prove that if the PIOP is knowledge sound, the commitment scheme is evaluation binding and black-box extractable, then the compiled (interactive) argument system has witness-extended emulation.

Theorem 4. *If the non-interactive polynomial commitment scheme PC is black-box extractable and evaluation-binding, and if the Rnds -round PIOP Π_{IOP} for $\mathcal{R}$ is knowledge-sound, then the argument Π described in Fig. 4 is a public-coin interactive argument system for $\mathcal{R}$ that has witness-extended emulation.*

Proof. Let $\mathbf{P}^*$, $\mathcal{A}$, $\mathcal{D}$ denote the adversaries from the WEE definition (Section 2.2). We construct a WEE emulator $\Pi.\text{Emu}$ for the argument system Π . $\Pi.\text{Emu}$ is defined in Fig. 5, having black-box rewindable access to $\mathbf{P}^*$.

We denote the state of the prover $\mathbf{P}^*$ (containing the state st from $\mathcal{A}$, $\mathbf{P}^*$'s internal configuration, and messages from V) at the beginning of the round r

$\Pi.\text{Emu}^{(P^*(p, ck, \mathbb{x}, st), V(vk_i, \mathbb{x}))}(p, ck, \mathbb{x})$

```

1 : colsn  $\leftarrow$  false;
2 : for  $r = 1$  to  $Rnds$  do //  $P^*$ 's initial state is  $st_0^{(1)} = (p, ck, \mathbb{x}, st)$ 
3 :   Run  $P^*(st_{r-1}^{(1)})$  and receive  $C_r$ ;
4 :   Sample extraction point  $\chi_r \leftarrow \mathcal{F}$ ;
5 :    $(\eta_{\chi_r}, \pi_{\chi_r}) \leftarrow P^*(st_{r-1}^{(1)}, \chi_r)$ ; //  $P^*$ 's new state is denoted  $st_{r-1}^{(2)}$ 
6 :    $tr_{PC} \leftarrow (C_r, \chi_r, \eta_{\chi_r}, \pi_{\chi_r})$ ;
7 :   if  $PC.V(ck, tr_{PC}) = 1$  then
8 :      $f_r(X) \leftarrow PC.Ext_{bb}^{P^*(st_{r-1}^{(1)}, \cdot)}(ck, tr_{PC})$ ;
9 :     if  $(C_r, f_r) \in \mathcal{R}_{ck}$  then
10 :      ext-fail $_r \leftarrow$  false;
11 :     else ext-fail $_r \leftarrow$  true; fi
12 :   fi
13 :   Sample coins $_r$  for  $IOP.V$ ;  $(\alpha_r, z_r, g_r) \leftarrow IOP.V(st_{r-1}^V, coins_r)$ ;
14 :    $(\eta_r, \pi_r) \leftarrow P^*(st_{r-1}^{(2)}, coins_r)$ ; //  $P^*$ 's new state is denoted  $st_r^{(1)}$ 
15 :    $st_r^V \leftarrow (st_{r-1}^V, coins_r, \eta_r, \pi_r)$ ;
16 :   if  $(PC.V(ck, tr_{PC}) = 1) \wedge \neg(\text{ext-fail}_r)$  then
17 :     Compute  $\tilde{\eta}_r \leftarrow (g_{r,1}(\alpha_{r,1}), \dots, g_{r,\ell_r}(\alpha_{r,\ell_r}))$ ;
18 :     if  $\eta_r \neq \tilde{\eta}_r$  then colsn  $\leftarrow$  true; fi
19 :   fi
20 : endfor
21 :  $tr \leftarrow (tr_1, \dots, tr_{Rnds})$ , where  $tr_i = (C_i, \chi_i, \eta_{\chi_i}, \pi_{\chi_i}, coins_i, \eta_i, \pi_i)$ ;
22 :  $tr_{IOP} \leftarrow (f_1, \alpha_1, z_1, g_1, \dots, f_{Rnds}, \alpha_{Rnds}, z_{Rnds}, g_{Rnds})$ ;
23 : if  $\bigvee_{i=1}^{Rnds} \text{ext-fail}_i$  then return  $(tr, \perp)$ ; fi
24 : if colsn then return  $(tr, \perp)$ ; fi
25 :  $w \leftarrow Ext(\mathbb{x}, tr_{IOP})$ , where  $Ext$  is the PIOP knowledge extractor.
26 : if  $(i, \mathbb{x}, w) \notin \mathcal{R}$  then return  $(tr, \perp)$ ; fi
27 : return  $(tr, w)$ ;

```

Fig. 5. Emulator $\Pi.\text{Emu}$ for the compiled argument Π .

subprotocol by $st_{r-1}^{(1)}$ and after receiving the challenge $\chi_r \leftarrow \mathcal{F}$ by $st_{r-1}^{(2)}$. Note that $\Pi.\text{Emu}$ has only an oracle access to P^* and does not see P^* 's state. We use the state notation only for showing that $\Pi.\text{Emu}$ rewinds the deterministic P^* to a particular state. We give an overview of $\Pi.\text{Emu}$ depicted in Fig. 5.

$\Pi.\text{Emu}$ black-box executes the prover's first round $P^*(st_0^{(1)})$ and gets a commitment C_1 to an unknown polynomial. The initial state is $st_0^{(1)} = (p, ck, \mathbb{x}, st)$, where st is the state information sent from $\mathcal{A}$. Then, $\Pi.\text{Emu}$ plays the argument's verifier, sampling $\chi_1 \leftarrow \mathcal{F}$ and running $P^*(st_0^{(1)}, \chi_1)$ to obtain an opening $(\eta_{\chi_1}, \pi_{\chi_1})$. If $\Pi.\text{Emu}$ received a valid opening for χ , it runs the black-box extractor $PC.Ext_{bb}$ of PC to obtain a polynomial f_1 , such that $(C_1, f_1) \in \mathcal{R}_{ck}$. We show later that Ext extracts a valid polynomial with overwhelming probability. This follows from the definition of black-box extractability (Definition 6), which can be invoked because χ_1 is an extraction point sampled from a superpolynomial

size set. Then, $\Pi.\text{Emu}$ samples coins_1 for IOP.V and uses them to compute evaluation points α_1 , challenges z_1 , and oracle labels g_1 that have to be evaluated on α_1 . It runs P^* on coins_1 to get openings (η_1, π_1) .

If f_1 was correctly extracted, $\Pi.\text{Emu}$ computes $\eta_{1,i} \leftarrow g_{1,i}(\alpha_{1,i})$, evaluating all the polynomials that the verifier indicated (g_1 may include a label for f_1 or indexer's polynomials) at the respective evaluation points. $\Pi.\text{Emu}$ checks if the openings returned by P^* are compatible with the one it can compute itself using f_1 . If so, then $\Pi.\text{Emu}$ has successfully simulated the first round of the PIOP, $(f_1, \alpha_1, z_1, g_1)$. Otherwise, if one of the openings is incompatible, we get a contradiction with the evaluation binding of PC, as we prove later. After updating the state of IOP.V , $\Pi.\text{Emu}$ proceeds to simulate the subsequent rounds similarly. Finally, the emulator runs the PIOP extractor on the PIOP transcript to recover the witness w . $\Pi.\text{Emu}$ outputs the argument's transcript and w .

Note that the transcript that $\Pi.\text{Emu}$ outputs is distributed as the one resulted from an honest protocol execution between P^* and V . Therefore, real and emulated transcripts are perfectly indistinguishable. Let $\mathbf{D}_0$ be the event “ $i \in \mathbb{I}(\mathcal{R}_{p,n}) \wedge \mathcal{D}(\text{tr}) = 1$ ” with the real transcript tr and $\mathbf{D}_1$ the same event when tr is emulated. Additionally, let $\mathbf{V}$ be the event that “ $V_{\text{check}}(\text{ck}, \text{tr}) = 1$ ” and $\mathbf{W}$ the event $(i, x, w) \in \mathcal{R}$ when the emulator outputs tr and w . Then,

$$\begin{aligned} \text{Adv}_{\text{Pgen}, \Pi, n, P^*, \mathcal{A}, \mathcal{D}}^{\text{wee}}(\lambda) &= \Pr[\mathbf{D}_0] - \Pr[\mathbf{D}_1 \wedge (\mathbf{V} \Rightarrow \mathbf{W})] \\ &= \Pr[\mathbf{D}_1] - \Pr[\mathbf{D}_1 \wedge (\neg \mathbf{V} \vee \mathbf{W})] \\ &= \Pr[\mathbf{D}_1 \wedge \mathbf{V} \wedge \neg \mathbf{W}] \leq \Pr[\mathbf{V} \wedge \neg \mathbf{W}] . \end{aligned}$$

Thus, to conclude the proof, we need to bound the probability that $\Pi.\text{Emu}$ computes a valid transcript but fails to extract a witness. This can be divided into the following disjoint events.

1. $\Pi.\text{Emu}$ computes a valid transcript, but on the line 23 in Fig. 5 it returns $(\text{tr}, \perp)$. Namely, there exists r , for which $\text{PC.Ext}_{\text{bb}}$ failed to extract a polynomial f_r such that $(C_r, f_r) \in \mathcal{R}_{\text{ck}}$, and consequently the corresponding variable ext-fail_r is set to **true**. We denote this event $\mathbf{F}_{\text{bbe}}^{(r)}$ (failure in the black box extraction). Let $\mathbf{F}_{\text{bbe}} = \mathbf{F}_{\text{bbe}}^{(1)} \vee \dots \vee \mathbf{F}_{\text{bbe}}^{(\text{Rnds})}$ be the event that extraction fails in any of the rounds.
2. $\Pi.\text{Emu}$ computes a valid transcript, but on the line 24 in Fig. 5 it returns $(\text{tr}, \perp)$. We denote the event by $\mathbf{F}_{\text{eb}}$ (failure in evaluation binding). In that case $\text{ext-fail}_r = \text{false}$ for all r , but $\text{colsn} = \text{true}$. The variable colsn is set to **true** if some evaluation $\eta_{r,\alpha}$ sent by P^* is different from $\tilde{\eta}_{r,\alpha} = g_{r,i}(\alpha_{r,i})$, where $g_{r,i}(X)$ is either a publicly-known indexer polynomial or one of the extracted polynomials and $\alpha_{r,i} \in \alpha_r$ is one of the query points.
3. $\Pi.\text{Emu}$ computes a valid transcript, but on the line 26 in Fig. 5 it returns $(\text{tr}, \perp)$. We denote this event by $\mathbf{F}_{\text{iop}}$ (failure in the PIOP extraction). In that case, tr_{IOP} compiled by the emulator is a valid PIOP transcript because polynomials have been successfully extracted, and there are no collisions, but the PIOP extractor still failed to extract the witness.

We prove that each of the three events happens with a negligible probability. We bound the first event by the probability of breaking black-box extractability. We show this first for a fixed round r .

Lemma 3. *There exists a expected PPT $\mathcal{A}_r$ such that $\Pr[\mathbf{F}_{\text{bbe}}^{(r)}] \leq \text{Adv}_{\text{Pgen,PC,Ext},n,\mathcal{A}_r,\mathbf{P}^*}^{\text{bbe}}(\lambda)$.*

Proof. We construct a reduction to the black-box extractability property of PC. Let $\mathcal{A}, \mathbf{P}^*$ be the adversaries in the WEE definition as above.

To construct a reduction, we need to construct a black-box extractability adversary $\mathcal{A}_r$ (we use $\mathbf{P}^*$ as the deterministic black-box extractability adversary). The adversary $\mathcal{A}_r$ runs $\Pi.\text{Emu}$ until the line 3 in r -th execution of **for** loop. That is, until the emulator gets C_r . At that point, $\mathcal{A}_r$ outputs $(C_r, \text{st}_{r-1}^{(1)})$.

We now plug $\mathcal{A}_r$ and $\mathbf{P}^*$ into the black-box extractability game. Let us recall the game. The adversary $\mathcal{A}_r(\text{ck})$ outputs $(C_r, \text{st}_{r-1}^{(1)})$ on a correctly sampled ck . Then the game picks $\chi \leftarrow \mathcal{F}_{\text{p}}$, sets $(\eta_{r,\chi}, \pi_{r,\chi}) \leftarrow \mathbf{P}^*(\text{st}_{r-1}^{(1)}, \chi)$ and defines $\text{tr}_{\text{PC}} \leftarrow (C_r, \chi, \eta_{r,\chi}, \pi_{r,\chi})$. Finally, $\text{PC.Ext}_{\text{bb}}^{\mathbf{P}^*(\text{st}_{r-1}^{(1)}, \cdot)}(\text{ck}, \text{tr}_{\text{PC}})$ outputs a polynomial $f_r(X)$. According to the definition, probability that $\text{PC.V}(\text{ck}, \text{tr}_{\text{PC}}) = 1$ and $(C_r, f_r) \notin \mathcal{R}_{\text{ck},\text{tr}}$ is bounded by $\text{Adv}_{\text{Pgen,PC,Ext},n,\mathcal{A}_r,\mathbf{P}^*}^{\text{bbe}}(\lambda)$.

By construction of $\mathcal{A}_r$ and $\mathbf{P}^*$ the latter event is implied by the event $\mathbf{F}_{\text{bbe}}$. Therefore, $\Pr[\mathbf{F}_{\text{bbe}}] \leq \text{Adv}_{\text{Pgen,PC,Ext},n,\mathcal{A}_r,\mathbf{P}^*}^{\text{bbe}}(\lambda)$. $\square$

From the above it follows that $\Pr[\mathbf{F}_{\text{bbe}}] \leq \sum_{r=1}^{\text{Rnds}} \text{Adv}_{\text{Pgen,PC,Ext},n,\mathcal{A}_r,\mathbf{P}^*}^{\text{bbe}}(\lambda) \leq \text{Rnds} \cdot \text{Adv}_{\text{Pgen,PC,Ext},n,\bar{\mathcal{A}},\mathbf{P}^*}^{\text{bbe}}(\lambda)$ for some $\bar{\mathcal{A}} \in \{\mathcal{A}_1, \dots, \mathcal{A}_{\text{Rnds}}\}$.

The second type of abortion can only happen with negligible probability due to the evaluation binding property (Section 2.1) of PC, which requires that it is computationally hard to open a commitment for two different evaluations at the same point. We prove it in the following lemma.

Lemma 4. *There exists a expected PPT $\mathcal{B}$ such that $\Pr[\mathbf{F}_{\text{eb}}] \leq \text{Adv}_{\text{Pgen,PC},n,\mathcal{B}}^{\text{evbind}}(\lambda)$.*

Proof. Recall that if $\Pi.\text{Emu}$ aborts on the line 24, then $\text{colsn} = \text{true}$, but $\text{ext-fail}_r = \text{false}$ for all $r \in [1, \text{Rnds}]$. It means that in each subprotocol the extraction of the polynomial was successful (for all r , $(C_r, f_r) \in \mathcal{R}_{\text{ck}}$), but for some r th subprotocol $\eta_{r,\alpha} \neq \tilde{\eta}_{r,\alpha} = g_{r,i}(\alpha_{r,i})$ for some $i \in [1, \ell_r]$ and a polynomial $g_{r,i}$, which is either one of the extracted polynomials or a publicly known indexer polynomial.

Given $\Pi.\text{Emu}$ as defined in Fig. 5, we construct an adversary $\mathcal{B}$ that breaks evaluation binding whenever the event $\mathbf{F}_{\text{eb}}$ happens. $\mathcal{B}$ gets (p, ck) as input and starts by internally running the WEE adversary $\mathcal{A}(\text{p}, \text{ck})$ to obtain (x, st) . After this $\mathcal{B}$ runs $\Pi.\text{Emu}$ until at some round r , $\mathbf{P}^*$ outputs (π_r, η_r) such that $\eta_r \neq \tilde{\eta}_r$. In this case, there exists $\alpha_{r,i} \in \alpha_r$ such that $\eta_{r,i} \neq \tilde{\eta}_{r,i} = g_{r,i}(\alpha_{r,i})$ for some $(C, g_{r,i}) \in \mathcal{R}_{\text{ck}}$. Here, C could be a commitment produced by an indexer (in that case $g_{r,i}(X)$ is publicly known) or one of the commitments sent by the prover (in

that case $(C, g_{r,i}) \in \mathcal{R}_{\text{ck}}$ since the extraction succeeded). $\mathcal{B}$ computes an opening proof $\tilde{\pi}_{r,i}$ using $\text{PC.Open}(\text{ck}, C, \alpha_{r,i}, g_{r,i})$ and outputs $(C, \alpha_{r,i}, \eta_{r,i}, \pi_{r,i}, \tilde{\eta}_{r,i}, \tilde{\pi}_{r,i})$.

The event $\mathbf{F}_{\text{eb}}$ implies the following:

- The argument’s verifier V accepts, thus $\text{PC.V}(\text{ck}, C, \alpha_{r,i}, \eta_{r,i}, \pi_{r,i}) = 1$.
- The extraction of polynomials was successful and thus $(C, g_{r,i}) \in \mathcal{R}_{\text{ck}}$. By the completeness property of the commitment scheme, $\text{PC.V}(\text{ck}, C, \alpha_{r,i}, \tilde{\eta}_{r,i}, \tilde{\pi}_{r,i}) = 1$.
- Since $\text{colsn} = \mathbf{true}$, $\eta_{r,i} \neq \tilde{\eta}_{r,i}$.

It follows that $\mathcal{B}$ breaks evaluation binding when the event $\mathbf{F}_{\text{eb}}$ happens. We get $\Pr[\mathbf{F}_{\text{eb}}] \leq \text{Adv}_{\text{Pgen,PC},n,\mathcal{B}}^{\text{evbind}}(\lambda)$. $\square$

Recall, $\mathbf{F}_{\text{iop}}$ is the event that tr_{IOP} is accepting, but the PIOP extractor in Step 25 returns w such that $(i, x, w) \notin \mathcal{R}$. We bound the probability of this event in the following lemma.

Lemma 5. *There exists an interactive algorithm $\mathcal{C}$ (potentially unbounded) such that $\Pr[\mathbf{F}_{\text{iop}}] \leq \text{Adv}_{\Pi_{\text{IOP}},\text{Ext},\mathcal{C}}^{\text{ks}}(\lambda)$.*

Proof. The knowledge soundness of PIOP holds for any fixed $i \in \mathbb{I}(\mathcal{R})$ and $x \in \{0,1\}^{\text{poly}(\lambda)}$. If it holds for any fixed input, then it will also hold when x and i are sampled. We sample $p \leftarrow \text{Pgen}(1^\lambda)$, $(\text{ck}, \text{tk}) \leftarrow \text{KGen}(p, n)$, and $(i, x, \text{st}) \leftarrow \mathcal{A}_1(\text{ck})$.

We will construct an adversary $\mathcal{C}$ against the knowledge soundness of PIOP that takes (i, x) as an input. The adversary $\mathcal{C}$ runs identically to the **for** loop in $\Pi.\text{Emu}$ except for two notable difference. First, instead of sampling coins_r by itself, it sends f_r to IOP.V and receives back (α_r, z_r, g_r) . It then computes coins_r corresponding to (α_r, z_r, g_r) . We assume that this is efficient since we deal with public coin protocols. Second, it aborts the protocol if either the verification of the commitment opening fails, extraction of a polynomial fails ($\text{ext-fail}_r = \mathbf{true}$), or there is a collision ($\text{colsn} = \mathbf{true}$). Complete details of $\mathcal{C}$ can be found in Fig. 6.

In case the extraction of f_r is successful in each round and there are no collisions, the resulting PIOP transcript tr_{IOP} is accepting. The probability of the PIOP extractor failing in this case corresponds precisely to the event $\mathbf{F}_{\text{iop}}$. Thus, $\Pr[\mathbf{F}_{\text{iop}}] \leq \text{Adv}_{\Pi_{\text{IOP}},\text{Ext},\mathcal{C}}^{\text{ks}}(\lambda)$. $\square$

By combining the results of all the lemmas above, we get that $\Pr[\mathbf{V} \wedge \neg \mathbf{W}] \leq \text{Rnds} \cdot \text{Adv}_{\text{Pgen,PC,Ext},n,\bar{\mathcal{A}},P^*}^{\text{bbe}}(\lambda) + \text{Adv}_{\text{Pgen,PC},n,\mathcal{B}}^{\text{evbind}}(\lambda) + \text{Adv}_{\Pi_{\text{IOP}},\text{Ext},\mathcal{C}}^{\text{ks}}(\lambda)$. Finally, we must consider that $\bar{\mathcal{A}}$ and $\mathcal{B}$ are expected PPT algorithms, but BBE and evaluation binding holds against strict PPT adversaries.

Both BBE and evaluation binding can be viewed as falsifiable assumptions. They can be written as an interaction between an efficient challenger and an adversary (see Section A for a formal definition). According to assumptions of our theorem, for any (strict) PPT $\mathcal{A}'$ and $\mathcal{B}'$, $\text{Adv}_{\text{Pgen,PC,Ext},n,\mathcal{A}',P^*}^{\text{bbe}}(\lambda)$ and $\text{Adv}_{\text{Pgen,PC},n,\mathcal{B}'}^{\text{evbind}}(\lambda)$ are negligible functions in λ . Thus, applying Lemma 6, $\text{Adv}_{\text{Pgen,PC,Ext},n,\bar{\mathcal{A}},P^*}^{\text{bbe}}(\lambda)$ and $\text{Adv}_{\text{Pgen,PC},n,\mathcal{B}}^{\text{evbind}}(\lambda)$ are also negligible. The claim of the theorem follows since $\text{Adv}_{\Pi_{\text{IOP}},\text{Ext},\mathcal{C}}^{\text{ks}}(\lambda)$ is negligible according to the knowledge soundness of PIOP. $\square$

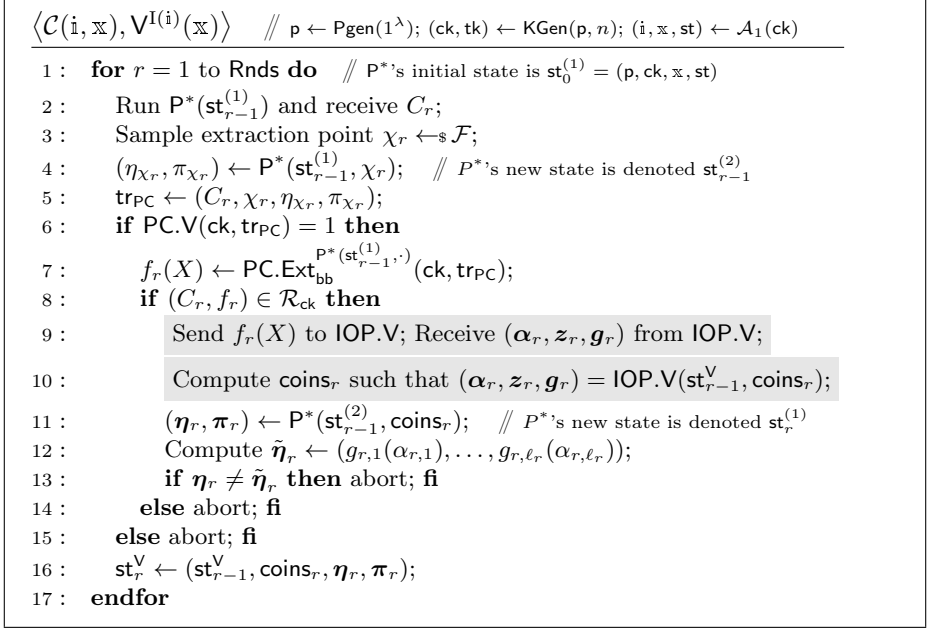


Fig. 6. The adversary $\mathcal{C}$ against knowledge soundness of PIOP. We have *highlighted* the main differences compared to $\Pi.\text{Emu}$.

As previously argued, KZG PCS has black-box extractability and evaluation binding under the ARSDH assumption. Thus, we get the following result.

Corollary 2. *If the ARSDH assumption holds and if a Rnds -round Polynomial IOP for $\mathcal{R}$ has negligible knowledge error, then the argument Π described in Fig. 4, instantiated with KZG PCS, is a public-coin interactive argument for $\mathcal{R}$ that has witness-extended emulation.*

Finally, if we use, for example, Plonk’s PIOP, then we obtain a public coin interactive argument with a constant proof size under a falsifiable assumption. The resulting argument retains Plonk’s original asymptotic efficiency features. Below, $O_\lambda(\cdot)$ is the common “Big-O” notation, but we ignore $\text{poly}(\lambda)$ factors.

Corollary 3. *Let N be the number of gates in an arithmetic circuit that describes a relation $\mathcal{R}$, and let $|x|$ denote the statement size. If the ARSDH assumption holds, then there exists a public coin argument system for $\mathcal{R}$ with $O_\lambda(1)$ proof size, $O_\lambda(N)$ SRS size, $O_\lambda(|x| + \log N)$ verifier’s running time, $O_\lambda(N \log N)$ prover’s running time, and a universal and updatable SRS.*

By applying the Fiat-Shamir transform, one can obtain a constant size SNARK, which is secure in the random oracle model assuming ARSDH. We recall the good, and often necessary practice, to instantiate the so-called strong Fiat-Shamir transform: hashing the common input with the current

portion of the transcript to obtain the following challenge. As noted in [BPW12,HLPT20,DMWG23] the usage of weak Fiat-Shamir transform, where only the transcript is hashed, can be exploited to break adaptive soundness of the resulting SNARK.

References

- ACK21. Thomas Attema, Ronald Cramer, and Lisa Kohl. A compressed Σ -protocol theory for lattices. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part II*, volume 12826 of *LNCS*, pages 549–579, Virtual Event, August 2021. Springer, Cham. doi:10.1007/978-3-030-84245-1_19.
- BB08. Dan Boneh and Xavier Boyen. Short signatures without random oracles and the SDH assumption in bilinear groups. *Journal of Cryptology*, 21(2):149–177, April 2008. doi:10.1007/s00145-007-9005-7.
- BBB⁺18. Benedikt Bünz, Jonathan Bootle, Dan Boneh, Andrew Poelstra, Pieter Wuille, and Greg Maxwell. Bulletproofs: Short proofs for confidential transactions and more. In *2018 IEEE Symposium on Security and Privacy*, pages 315–334. IEEE Computer Society Press, May 2018. doi:10.1109/SP.2018.00020.
- BBHR18. Eli Ben-Sasson, Iddo Bentov, Yinon Horesh, and Michael Riabzev. Fast reed-solomon interactive oracle proofs of proximity. In Ioannis Chatzigiannakis, Christos Kaklamanis, Dániel Marx, and Donald Sannella, editors, *ICALP 2018*, volume 107 of *LIPIcs*, pages 14:1–14:17. Schloss Dagstuhl, July 2018. doi:10.4230/LIPIcs.ICALP.2018.14.
- BCG⁺14. Eli Ben-Sasson, Alessandro Chiesa, Christina Garman, Matthew Green, Ian Miers, Eran Tromer, and Madars Virza. Zerocash: Decentralized anonymous payments from bitcoin. In *2014 IEEE Symposium on Security and Privacy*, pages 459–474. IEEE Computer Society Press, May 2014. doi:10.1109/SP.2014.36.
- BCGT13. Eli Ben-Sasson, Alessandro Chiesa, Daniel Genkin, and Eran Tromer. On the concrete efficiency of probabilistically-checkable proofs. In Dan Boneh, Tim Roughgarden, and Joan Feigenbaum, editors, *STOC 2013*, pages 585–594, Palo Alto, CA, USA, June 1–4, 2013. ACM Press. doi:10.1145/2488608.2488681.
- BCPR14. Nir Bitansky, Ran Canetti, Omer Paneth, and Alon Rosen. On the existence of extractable one-way functions. In David B. Shmoys, editor, *46th ACM STOC*, pages 505–514. ACM Press, May / June 2014. doi:10.1145/2591796.2591859.
- BDFG20. Dan Boneh, Justin Drake, Ben Fisch, and Ariel Gabizon. Efficient polynomial commitment schemes for multiple points and polynomials. Cryptology ePrint Archive, Report 2020/081, 2020. URL: <https://eprint.iacr.org/2020/081>.
- BFS20. Benedikt Bünz, Ben Fisch, and Alan Szepieniec. Transparent SNARKs from DARK compilers. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 677–706. Springer, Cham, May 2020. doi:10.1007/978-3-030-45721-1_24.

- BMM⁺21. Benedikt Bünz, Mary Maller, Pratyush Mishra, Nirvan Tyagi, and Psi Vesely. Proofs for inner pairing products and applications. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part III*, volume 13092 of *LNCS*, pages 65–97. Springer, Cham, December 2021. doi:10.1007/978-3-030-92078-4_3.
- BPW12. David Bernhard, Olivier Pereira, and Bogdan Warinschi. How not to prove yourself: Pitfalls of the Fiat-Shamir heuristic and applications to Helios. In Xiaoyun Wang and Kazue Sako, editors, *ASIACRYPT 2012*, volume 7658 of *LNCS*, pages 626–643. Springer, Berlin, Heidelberg, December 2012. doi:10.1007/978-3-642-34961-4_38.
- CDS94. Ronald Cramer, Ivan Damgård, and Berry Schoenmakers. Proofs of partial knowledge and simplified design of witness hiding protocols. In Yvo Desmedt, editor, *CRYPTO'94*, volume 839 of *LNCS*, pages 174–187. Springer, Berlin, Heidelberg, August 1994. doi:10.1007/3-540-48658-5_19.
- CFF⁺21. Matteo Campanelli, Antonio Faonio, Dario Fiore, Anaïs Querol, and Hadrián Rodríguez. Lunar: A toolbox for more efficient universal and updatable zkSNARKs and commit-and-prove extensions. In Mehdi Tibouchi and Huaxiong Wang, editors, *ASIACRYPT 2021, Part III*, volume 13092 of *LNCS*, pages 3–33. Springer, Cham, December 2021. doi:10.1007/978-3-030-92078-4_1.
- CGKS23. Matteo Campanelli, Chaya Ganesh, Hamidreza Khoshakhlagh, and Janno Siim. Impossibilities in succinct arguments: Black-box extraction and more. In Nadia El Mrabet, Luca De Feo, and Sylvain Duquesne, editors, *AFRICACRYPT 23*, volume 14064 of *LNCS*, pages 465–489. Springer, Cham, July 2023. doi:10.1007/978-3-031-37679-5_20.
- CHM⁺20. Alessandro Chiesa, Yuncong Hu, Mary Maller, Pratyush Mishra, Psi Vesely, and Nicholas P. Ward. Marlin: Preprocessing zkSNARKs with universal and updatable SRS. In Anne Canteaut and Yuval Ishai, editors, *EUROCRYPT 2020, Part I*, volume 12105 of *LNCS*, pages 738–768. Springer, Cham, May 2020. doi:10.1007/978-3-030-45721-1_26.
- DMWG23. Quang Dao, Jim Miller, Opal Wright, and Paul Grubbs. Weak fiat-shamir attacks on modern proof systems. In *44th IEEE Symposium on Security and Privacy, SP 2023, San Francisco, CA, USA, May 21-25, 2023*, pages 199–216. IEEE, 2023. doi:10.1109/SP46215.2023.10179408.
- FKL18. Georg Fuchsbauer, Eike Kiltz, and Julian Loss. The algebraic group model and its applications. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part II*, volume 10992 of *LNCS*, pages 33–62. Springer, Cham, August 2018. doi:10.1007/978-3-319-96881-0_2.
- FS87. Amos Fiat and Adi Shamir. How to prove yourself: Practical solutions to identification and signature problems. In Andrew M. Odlyzko, editor, *CRYPTO'86*, volume 263 of *LNCS*, pages 186–194. Springer, Berlin, Heidelberg, August 1987. doi:10.1007/3-540-47721-7_12.
- GGPR13. Rosario Gennaro, Craig Gentry, Bryan Parno, and Mariana Raykova. Quadratic span programs and succinct NIZKs without PCs. In Thomas Johansson and Phong Q. Nguyen, editors, *EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 626–645. Springer, Berlin, Heidelberg, May 2013. doi:10.1007/978-3-642-38348-9_37.
- GKM⁺18. Jens Groth, Markulf Kohlweiss, Mary Maller, Sarah Meiklejohn, and Ian Miers. Updatable and universal common reference strings with applications

- to zk-SNARKs. In Hovav Shacham and Alexandra Boldyreva, editors, *CRYPTO 2018, Part III*, volume 10993 of *LNCS*, pages 698–728. Springer, Cham, August 2018. doi:10.1007/978-3-319-96878-0_24.
- GR19. Alonso González and Carla Ràfols. Shorter pairing-based arguments under standard assumptions. In Steven D. Galbraith and Shihō Moriai, editors, *ASIACRYPT 2019, Part III*, volume 11923 of *LNCS*, pages 728–757. Springer, Cham, December 2019. doi:10.1007/978-3-030-34618-8_25.
- Gro16. Jens Groth. On the size of pairing-based non-interactive arguments. In Marc Fischlin and Jean-Sébastien Coron, editors, *EUROCRYPT 2016, Part II*, volume 9666 of *LNCS*, pages 305–326. Springer, Berlin, Heidelberg, May 2016. doi:10.1007/978-3-662-49896-5_11.
- GW11. Craig Gentry and Daniel Wichs. Separating succinct non-interactive arguments from all falsifiable assumptions. In Lance Fortnow and Salil P. Vadhan, editors, *43rd ACM STOC*, pages 99–108. ACM Press, June 2011. doi:10.1145/1993636.1993651.
- GWC19. Ariel Gabizon, Zachary J. Williamson, and Oana Ciobotaru. PLONK: Permutations over Lagrange-bases for oecumenical noninteractive arguments of knowledge. Cryptology ePrint Archive, Report 2019/953, 2019. URL: <https://eprint.iacr.org/2019/953>.
- HLPT20. Thomas Haines, Sarah Jamie Lewis, Olivier Pereira, and Vanessa Teague. How not to prove your election outcome. In *2020 IEEE Symposium on Security and Privacy*, pages 644–660. IEEE Computer Society Press, May 2020. doi:10.1109/SP40000.2020.00048.
- JT20. Joseph Jaeger and Stefano Tessaro. Expected-time cryptography: Generic techniques and applications to concrete soundness. In Rafael Pass and Krzysztof Pietrzak, editors, *TCC 2020, Part III*, volume 12552 of *LNCS*, pages 414–443. Springer, Cham, November 2020. doi:10.1007/978-3-030-64381-2_15.
- KMS⁺16. Ahmed E. Kosba, Andrew Miller, Elaine Shi, Zikai Wen, and Charalampos Papamanthou. Hawk: The blockchain model of cryptography and privacy-preserving smart contracts. In *2016 IEEE Symposium on Security and Privacy*, pages 839–858. IEEE Computer Society Press, May 2016. doi:10.1109/SP.2016.55.
- KZG10. Aniket Kate, Gregory M. Zaverucha, and Ian Goldberg. Constant-size commitments to polynomials and their applications. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 177–194. Springer, Berlin, Heidelberg, December 2010. doi:10.1007/978-3-642-17373-8_11.
- Lee21. Jonathan Lee. Dory: Efficient, transparent arguments for generalised inner products and polynomial commitments. In Kobbi Nissim and Brent Waters, editors, *TCC 2021, Part II*, volume 13043 of *LNCS*, pages 1–34. Springer, Cham, November 2021. doi:10.1007/978-3-030-90453-1_1.
- Lin01. Yehuda Lindell. Parallel coin-tossing and constant-round secure two-party computation. In Joe Kilian, editor, *CRYPTO 2001*, volume 2139 of *LNCS*, pages 171–189. Springer, Berlin, Heidelberg, August 2001. doi:10.1007/3-540-44647-8_10.
- Lip12. Helger Lipmaa. Progression-free sets and sublinear pairing-based non-interactive zero-knowledge arguments. In Ronald Cramer, editor, *TCC 2012*, volume 7194 of *LNCS*, pages 169–189. Springer, Berlin, Heidelberg, March 2012. doi:10.1007/978-3-642-28914-9_10.

- LM19. Russell W. F. Lai and Giulio Malavolta. Subvector commitments with application to succinct arguments. In Alexandra Boldyreva and Daniele Micciancio, editors, *CRYPTO 2019, Part I*, volume 11692 of *LNCS*, pages 530–560. Springer, Cham, August 2019. doi:10.1007/978-3-030-26948-7_19.
- LPS23. Helger Lipmaa, Roberto Parisella, and Janno Siim. Algebraic Group Model with Oblivious Sampling. In Guy Rothblum and Hoeteck Wee, editors, *TCC 2023 (4)*, volume 14372 of *LNCS*, pages 363–392, Taipei, Taiwan, Nov 29–Dec 2 2023. Springer, Cham. doi:10.1007/978-3-031-48624-1_14.
- LSZ22. Helger Lipmaa, Janno Siim, and Michal Zajac. Counting vampires: From univariate sumcheck to updatable ZK-SNARK. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part II*, volume 13792 of *LNCS*, pages 249–278. Springer, Cham, December 2022. doi:10.1007/978-3-031-22966-4_9.
- Nao03. Moni Naor. On cryptographic assumptions and challenges (invited talk). In Dan Boneh, editor, *CRYPTO 2003*, volume 2729 of *LNCS*, pages 96–109. Springer, Berlin, Heidelberg, August 2003. doi:10.1007/978-3-540-45146-4_6.
- PHGR13. Bryan Parno, Jon Howell, Craig Gentry, and Mariana Raykova. Pinocchio: Nearly practical verifiable computation. In *2013 IEEE Symposium on Security and Privacy*, pages 238–252. IEEE Computer Society Press, May 2013. doi:10.1109/SP.2013.47.
- PST13. Charalampos Papamanthou, Elaine Shi, and Roberto Tamassia. Signatures of correct computation. In Amit Sahai, editor, *TCC 2013*, volume 7785 of *LNCS*, pages 222–242. Springer, Berlin, Heidelberg, March 2013. doi:10.1007/978-3-642-36594-2_13.
- RZ21. Carla Ràfols and Arantxa Zapico. An algebraic framework for universal and updatable SNARKs. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part I*, volume 12825 of *LNCS*, pages 774–804, Virtual Event, August 2021. Springer, Cham. doi:10.1007/978-3-030-84242-0_27.
- Sho97. Victor Shoup. Lower bounds for discrete logarithms and related problems. In Walter Fumy, editor, *EUROCRYPT’97*, volume 1233 of *LNCS*, pages 256–266. Springer, Berlin, Heidelberg, May 1997. doi:10.1007/3-540-69053-0_18.
- Sta21. StarkWare. ethSTARK documentation. Cryptology ePrint Archive, Report 2021/582, 2021. URL: <https://eprint.iacr.org/2021/582>.
- SZ20. Alan Szepieniec and Yuncong Zhang. Polynomial IOPs for Linear Algebra Relations. Technical Report 2020/1022, IACR, August 24, 2020. Last checked modification from June 9, 2021. URL: <https://ia.cr/2020/1022>.
- TAB⁺20. Alin Tomescu, Ittai Abraham, Vitalik Buterin, Justin Drake, Dankrad Feist, and Dmitry Khovratovich. Aggregatable subvector commitments for stateless cryptocurrencies. In Clemente Galdi and Vladimir Kolesnikov, editors, *SCN 20*, volume 12238 of *LNCS*, pages 45–64. Springer, Cham, September 2020. doi:10.1007/978-3-030-57990-6_3.
- Zha22. Mark Zhandry. To label, or not to label (in generic groups). In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 66–96. Springer, Cham, August 2022. doi:10.1007/978-3-031-15982-4_3.
- ZZ23. Cong Zhang and Mark Zhandry. The relationship between idealized models under computationally bounded adversaries. In Jian Guo and

Ron Steinfeld, editors, *ASIACRYPT 2023, Part VI*, volume 14443 of *LNCS*, pages 390–419. Springer, Singapore, December 2023. doi:10.1007/978-981-99-8736-8_13.

ZZK22. Cong Zhang, Hong-Sheng Zhou, and Jonathan Katz. An analysis of the algebraic group model. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part IV*, volume 13794 of *LNCS*, pages 310–322. Springer, Cham, December 2022. doi:10.1007/978-3-031-22972-5_11.

A Expected versus Strict PPT Adversaries

In this section, we prove that if a falsifiable assumption is secure for strict PPT adversaries, it is also secure for expected PPT adversaries. Recall the definition of a falsifiable cryptographic assumption [GW11].

Definition 8. *A falsifiable cryptographic assumption consists of a PPT interactive challenger C and a constant $\epsilon \in [0, 1]$. On a security parameter λ , $C(1^\lambda)$ interacts with an adversary $\mathcal{A}(1^\lambda)$ and C outputs either 0 or 1. We say that $\mathcal{A}$ wins if the output is 1.*

The assumption related to (C, ϵ) states that for any efficient (either PPT or expected PPT) $\mathcal{A}$, we have $\Pr[\mathcal{A}(1^\lambda) \text{ wins } C(1^\lambda)] - \epsilon \approx_\lambda 0$, where the probability is over random coins of $\mathcal{A}$ and C .

For example, in decisional assumptions $\epsilon = 1/2$ and in search assumptions $\epsilon = 0$.

Lemma 6. *Let (C, ϵ) be a falsifiable assumption. If for all PPT $\mathcal{A}$, $\Pr[\mathcal{A}(1^\lambda) \text{ wins } C(1^\lambda)] - \epsilon \approx_\lambda 0$, then for any expected PPT $\mathcal{A}'$, $\Pr[\mathcal{A}'(1^\lambda) \text{ wins } C(1^\lambda)] - \epsilon \approx_\lambda 0$.*

Proof. Let $\varepsilon(\lambda)$ be a negligible function, such that $\Pr[\mathcal{A}(1^\lambda) \text{ wins } C(1^\lambda)] \leq \epsilon + \varepsilon(\lambda)$ for each PPT $\mathcal{A}$. Let T be the running time of $\mathcal{A}'$. Let $\mathcal{A}'$ be an expected PPT adversary and let T denote its running time. Suppose the expected running time $\mathbb{E}[T]$ is asymptotically bounded by $T_{\mathbb{E}}(\lambda)$ for some polynomial $T_{\mathbb{E}}(X)$.

By the Markov inequality, for each polynomial g , $\Pr[T \geq T_{\mathbb{E}}(\lambda)g(\lambda)] \leq \mathbb{E}[T]/(T_{\mathbb{E}}(\lambda) \cdot g(\lambda)) \leq 1/g(\lambda)$. Let $\mathcal{B}_{\mathcal{A}',g}$ be a (strict) PPT adversary that runs precisely as $\mathcal{A}$, except it halts and outputs $\perp$ if $\mathcal{A}$ has not terminated in $T_{\mathbb{E}}(\lambda)g(\lambda)$ steps. Conditioned on the event $T < T_{\mathbb{E}}(\lambda)g(\lambda)$, the output of $\mathcal{A}'$ and $\mathcal{B}_{\mathcal{A}',g}$ are equally distributed. We have

$$\begin{aligned} \Pr[\mathcal{A}'(1^\lambda) \text{ wins } C(1^\lambda)] &\leq \Pr[\mathcal{B}_{\mathcal{A}',g}(1^\lambda) \text{ wins } C(1^\lambda)] + \Pr[T \geq T_{\mathbb{E}}(\lambda)g(\lambda)] \\ &\leq \epsilon + \varepsilon(\lambda) + 1/g(\lambda) . \end{aligned}$$

Since the previous holds for any polynomial g , then $\Pr[\mathcal{A}'(1^\lambda) \text{ wins } C(1^\lambda)] - \epsilon - \varepsilon'(\lambda)$ is negligible. Moreover, since $\varepsilon'(\lambda)$ is negligible, and a sum of two negligible functions is negligible, then $\Pr[\mathcal{A}'(1^\lambda) \text{ wins } C(1^\lambda)] - \epsilon \approx_\lambda 0$. $\square$

The previous lemma does not establish any tight relationship between the advantage of (strict) PPT adversaries and the advantage of expected PPT ones,

besides the fact that if one of them is negligible, so is the other. The latter is not ideal since we reduce the advantage of (strict) PPT adversaries to concrete computational assumptions. To state concrete security parameters of our SNARK in Section 7, we want to reduce the advantage of expected PPT adversaries to the ARSDH assumption (that holds against strict PPT adversaries). We leave this as an important open question. We hope the techniques from [JT20] can be generalized to answer the previous question.

B AGMOS proof of ARSDH

While S -RSDH is a known, falsifiable, standard-looking assumption, adaptive ARSDH is possibly stronger. Moreover, [GR19] only gave an informal GGM (*without* oblivious sampling) argument why S -RSDH is secure. To gain further confidence, we prove that ARSDH holds in the AGMOS [LPS23] under the PDL [Lip12] and Tensor Oracle FindRep (TOFR) [LPS23] assumptions. From this, it follows that ARSDH holds in the AGM under the PDL assumption. We emphasize that the security of our constructions (the KZG commitment and the resulting SNARKs) depend only on falsifiable ARSDH and not on the AGM(OS). We use AGMOS only as a sanity check.

Lipmaa et al. [LPS23] recently defined AGMOS (AGM with oblivious sampling). AGMOS is more realistic than AGM since AGMOS adversaries are given an additional power of sampling group elements without knowing their discrete logarithms. As shown in [LPS23], certain uses of KZG are secure in AGM but not in AGMOS. Since AGMOS is a new model, we will give a longer description of AGMOS and TOFR (an underlying security assumption); our description follows closely [LPS23].

Fix a pairing description $\mathbf{p} \leftarrow \text{Pgen}(1^\lambda)$. Let $\mathcal{EF}_{\mathbf{p},\iota}$ be a set of (polynomially many) functions $\mathbb{F} \rightarrow \mathbb{G}_\iota$. Let $\mathcal{DF}_{\mathbf{p}}$ be a family of distributions over $\mathbb{F}$. We introduce two oracles $\mathcal{O}_1$ and $\mathcal{O}_2$, one for each group $\mathbb{G}_1$ and $\mathbb{G}_2$. Let $\mathcal{O} = (\mathcal{O}_1, \mathcal{O}_2)$. The i th query (E, D) to $\mathcal{O}_i$ consists of a function $E \in \mathcal{EF}_{\mathbf{p},\iota}$ and a distribution $D \in \mathcal{DF}_{\mathbf{p}}$. The oracle samples a random field element $s_i \leftarrow_\$ D$ and returns $[\mathbf{q}_{\iota,i}]_\iota \leftarrow E(s_i)$ and s_i .

We will denote the adversary's initial input (e.g., input from the challenger) in $\mathbb{G}_\iota$ by $[\mathbf{x}_\iota]_\iota$. We assume $[\mathbf{x}_\iota]_\iota$ always includes $[1]_\iota$. Let $\mathbf{x} = ([\mathbf{x}_1]_1, [\mathbf{x}_2]_2)$. The adversary's view consists of all group elements that the adversary has seen up to the given moment. This includes the adversary's initial input, elements sent by other parties during the interaction, and oracle answers.

Let $\mathcal{O}$ be as above. We require that for any PPT oracle adversary $\mathcal{A}^\mathcal{O}$, there exists a (non-uniform) PPT extractor $\text{Ext}_{\mathcal{A}}^\mathcal{O}$, such that: if $\mathcal{A}^\mathcal{O}(\mathbf{x})$ outputs a vector of group elements $[\mathbf{y}]_\iota$, on input $\mathbf{x} = ([\mathbf{x}_1]_1, [\mathbf{x}_2]_2)$, then with an overwhelming probability, $\text{Ext}_{\mathcal{A}}^\mathcal{O}$ outputs field-element matrices $\boldsymbol{\gamma}$, $\boldsymbol{\delta}$, and $[\mathbf{q}_\iota]_\iota$ ($\mathcal{O}_\iota$'s answer vector), such that

$$\mathbf{y} = \boldsymbol{\gamma}^\top \mathbf{x}_\iota + \boldsymbol{\delta}^\top \mathbf{q}_\iota \quad . \quad (5)$$

Definition 9 (AGMOS). Let $\mathcal{EF} = \{\mathcal{EF}_{\mathbf{p},\iota}\}$ be a collection of functions. Let $\mathcal{DF} = \{\mathcal{DF}_{\mathbf{p}}\}$ be a family of distributions. A PPT algorithm $\mathcal{A}$ is an $(\mathcal{EF}, \mathcal{DF})$ -

$\mathcal{O}_\iota(E, D)$
if $E \notin \mathcal{EF}_{p,\iota} \vee D \notin \mathcal{DF}_p$ then return $\perp$; fi
$s \leftarrow D$; $[\mathbf{q}]_\iota \leftarrow E(s)$; return $([\mathbf{q}]_\iota, s)$;

Fig. 7. The description of the oblivious sampling oracle $\mathcal{O}_\iota$, where $\iota \in \{1, 2\}$.

AGMOS adversary for Pgen if there exists a PPT extractor $\text{Ext}_{\mathcal{A}}$, such that for any $\mathbb{X} = (\mathbb{X}_1, \mathbb{X}_2)$, $\text{Adv}_{\text{Pgen}, \mathcal{EF}, \mathcal{DF}, \mathcal{A}, \text{Ext}_{\mathcal{A}}}^{\text{agmos}}(\lambda) :=$

$$\Pr \left[\begin{array}{l} \mathbf{y}_1 \neq \gamma_1^\top \mathbb{X}_1 + \delta_1^\top \mathbf{q}_{11} \vee \\ \mathbf{y}_2 \neq \gamma_2^\top \mathbb{X}_2 + \delta_2^\top \mathbf{q}_{12} \end{array} \middle| \begin{array}{l} \mathbf{p} \leftarrow \text{Pgen}(1^\lambda); r \leftarrow \text{RND}_\lambda(\mathcal{A}); \\ ([\mathbf{y}_1]_1, [\mathbf{y}_2]_2) \leftarrow \mathcal{A}^\mathcal{O}(\mathbf{p}, \mathbb{X}; r); \\ (\gamma_\iota, \delta_\iota, [\mathbf{q}_\iota]_\iota)_{\iota=1}^2 \leftarrow \text{Ext}_{\mathcal{A}}^\mathcal{O}(\mathbf{p}, \mathbb{X}; r) \end{array} \right] \approx_\lambda 0 .$$

$\mathcal{O}$ is the non-programmable oracle depicted in Fig. 7. Here, $[\mathbf{q}_\iota]_\iota$ is required to be the tuple of elements output by $\mathcal{O}_\iota$. We denote by i_ι the number of $\mathcal{O}_\iota$ calls.

Many AGMOS proofs rely on the following assumption.

Definition 10 (TOFR, [LPS23]). Let $\mathcal{EF}$ be some family of function and $\mathcal{DF}$ a family of distributions. We say that Pgen is $(\mathcal{EF}, \mathcal{DF})$ -TOFR (Tensor Oracle FindRep) secure if for any PPT $\mathcal{A}$, $\text{Adv}_{\text{Pgen}, \mathcal{A}}^{\text{tofr}}(\lambda) :=$

$$\Pr \left[\mathbf{v} \neq \mathbf{0} \wedge \mathbf{v}^\top \cdot \begin{pmatrix} 1 \\ \mathbf{q}_1 \\ \mathbf{q}_2 \\ \mathbf{q}_1 \otimes \mathbf{q}_2 \end{pmatrix} = 0 \middle| \mathbf{p} \leftarrow \text{Pgen}(1^\lambda); \mathbf{v} \leftarrow \mathcal{A}^\mathcal{O}(\mathbf{p}) \right] \approx_\lambda 0 .$$

Here, $\mathcal{O}$, $\mathbf{q}_1$, and $\mathbf{q}_2$ are as in Definition 9.

Theorem 5. Let $n \in \text{poly}(\lambda)$ and Pgen be a bilinear-group generator. If $(n, 2)$ -PDL and $(\mathcal{EF}, \mathcal{DF})$ -TOFR hold, $(n+1)$ -ARSDH holds in the $(\mathcal{EF}, \mathcal{DF})$ -AGMOS.

Proof (Sketch). Let $\mathcal{A}$ be an ARSDH adversary that with some non-negligible probability outputs $(\mathcal{S} = \{\alpha_i\}_{i=1}^{n+1}, [g, \varphi]_1)$ such that $|\mathcal{S}| = n+1$, $g \neq 0$, and $[g]_1 \bullet [1]_2 = [\varphi]_1 \bullet [\mathbf{Z}_{\mathcal{S}}(\sigma)]_2$. By using the algebraic representation provided the algebraic adversary, we obtain polynomials $g(X, \mathbf{Q})$ and $\varphi(X, \mathbf{Q})$, such that $[g]_1 = [g(\sigma, \mathbf{q})]_1$ and $[\varphi]_1 = [\varphi(\sigma, \mathbf{q})]_1$. Moreover, $g(X, \mathbf{Q}) = g_1(X) + \mathbf{g}_2^\top \mathbf{Q}$ and $\varphi(X, \mathbf{Q}) = \varphi_1(X) + \varphi_2^\top \mathbf{Q}$ for some known coefficients. Define the verification polynomial

$$V(X, \mathbf{Q}) := g(X, \mathbf{Q}) - \varphi(X, \mathbf{Q}) \mathbf{Z}_{\mathcal{S}}(X) = g_1(X) + \mathbf{g}_2^\top \mathbf{Q} - (\varphi_1(X) + \varphi_2^\top \mathbf{Q}) \mathbf{Z}_{\mathcal{S}}(X) .$$

Clearly, $V(X, \mathbf{Q}) = V^h(X) + V^t(X, \mathbf{Q})$ for $V^h(X) = g_1(X) - \varphi_1(X) \mathbf{Z}_{\mathcal{S}}(X)$ and $V^t(X, \mathbf{Q}) = \mathbf{g}_2^\top \mathbf{Q} - \varphi_2^\top \mathbf{Q} \mathbf{Z}_{\mathcal{S}}(X)$. Here, V^h does not depend on $\mathbf{Q}$ while each term of V^t depends on some Q_k .

Importantly, we know all the coefficients of $V(X, \mathbf{Q})$. Since the verifier accepts, $V(\sigma, \mathbf{q}) = 0$. Following [LPS23], we next consider separately four cases. The probability that $\mathcal{A}$ succeeds is bound by the sum of the probabilities it succeeds in all four cases.

Case A ($V(X, \mathbf{Q}) = 0$ as a polynomial). Then, $g_{2k}Q_k = \varphi_{2k}Q_k\mathbf{Z}_S(X)$ for each k . Since each Q_k is an independent variable, we get that $g_{2k} = \varphi_{2k} = 0$. Thus, $g(X, \mathbf{Q}) = g_1(X)$ and $\varphi(X, \mathbf{Q}) = \varphi_1(X)$. Then, $\mathbf{Z}_S(X) \mid g_1(X)$ and thus each α_i is a root of $g_1(X)$. By assumption, $\alpha_i \neq \alpha_j$. Hence, $g_1(X)$ has at least $n+1$ roots. Since $\deg g_1 \leq n$, $g_1(X)$ is a zero polynomial. Contradiction with $g_1(\sigma) = g \neq 0$.

Case X.1 ($V(X, \mathbf{Q}) \neq 0$ but $V^t(X, \mathbf{Q}) = 0$). Then, $V^h(X)$ is a non-zero polynomial with a known root σ . In this case, we construct a PDL adversary $\mathcal{B}$ that on input $\text{ck} = ([(\sigma^i)_{i=0}^n]_1, [1, \sigma]_2)$ does the following. It sends ck to $\mathcal{A}$ and then obtains $(\mathcal{S}, [g, \varphi]_1)$. It uses the AGM extractor to obtain all coefficients of $g(X)$ and $\varphi(X)$. It factors $V^h(X)$, finding its at most $2n+1$ roots x_i . It then tests by brute force for each i whether $[\sigma]_1 = [x_i]_1$, and returns the value $\sigma \leftarrow x_j$ for which $[\sigma]_1 = [x_j]_1$ holds. Clearly, $\mathcal{B}$ succeeds in computing σ with the same probability as $\mathcal{A}$ succeeds in breaking the ARSDH.

Case X.2 ($V^t(X, \mathbf{Q}) \neq 0$ but $V^t(\sigma, \mathbf{Q}) = 0$). Recall

$$V^t(X, \mathbf{Q}) = \sum (g_{2k} - \varphi_{2k}\mathbf{Z}_S(X))Q_k = \sum \beta_k(X)Q_k ,$$

where $\beta_k(X) := g_{2k} - \varphi_{2k}\mathbf{Z}_S(X)$. In Case X.2, we have that $\beta_k(X) \neq 0$ but $\beta_k(\sigma) = 0$ for some k . Hence, σ is the root of the non-zero polynomial $\varphi_{2k}\mathbf{Z}_S(X) - g_{2k}$. One can construct a different PDL adversary that computes σ .

Case Q ($V^t(\sigma, \mathbf{Q}) \neq 0$ but $V^t(\sigma, \mathbf{Q}) = 0$). We can construct a TOFR adversary that samples σ , constructs ck , and after interacting with $\mathcal{A}$, returns

$$\mathbf{v} = \begin{pmatrix} V^h(\sigma) \\ \beta(\sigma) \\ \mathbf{0} \end{pmatrix} .$$

Since $\mathbf{v}$ is a non-zero vector and

$$\mathbf{v}^\top \cdot \begin{pmatrix} 1 \\ \mathbf{q}_1 \\ \mathbf{q}_2 \\ \mathbf{q}_1 \otimes \mathbf{q}_2 \end{pmatrix} = 0 ,$$

$\mathcal{B}$ breaks the TOFR assumption.

Summarizing, if TOFR and PDL hold, ARSDH is secure in the AGMOS. $\square$

C The Relation of BBE and Evaluation Binding

It is always important to understand how novel security notions related to each other and already known security notions. We prove that if black-box extractability holds, then (a variant of) evaluation binding follows from (and, thus, is equivalent to) binding. We only obtain a variant (see Definition 11) due to the same issue as with the definition of black-box extractability itself: namely, α has to be chosen randomly and not by the adversary.

Definition 11. A polynomial commitment scheme PC is non-adaptively evaluation-binding for Pgen , if for any λ , $n \in \text{poly}(\lambda)$, and PPT adversary $\mathcal{A}$, $\text{DPT } \text{P}^*$, and $\mathcal{F}_p \subseteq \mathbb{F}$ of size $\lambda^{\omega(1)}$, $\text{Adv}_{\text{Pgen}, \text{PC}, n, \mathcal{A}, \text{P}^*}^{\text{naeb}}(\lambda) :=$

$$\Pr \left[\begin{array}{l} V(\text{ck}, C, \alpha, \eta, \pi) = 1 \wedge \\ V(\text{ck}, C, \alpha, \eta', \pi') = 1 \wedge \\ \eta \neq \eta' \end{array} \middle| \begin{array}{l} \mathbf{p} \leftarrow \text{Pgen}(1^\lambda); (\text{ck}, \text{tk}) \leftarrow \text{KGen}(\mathbf{p}, n); \\ (C, \text{st}) \leftarrow \mathcal{A}(\mathbf{p}, \text{ck}); \alpha \leftarrow_{\mathcal{S}} \mathcal{F}_p; \\ (\eta, \pi, \eta', \pi') \leftarrow \text{P}^*(\text{st}, \alpha) \end{array} \right] \approx_{\lambda} 0 .$$

We also recall the standard binding definition for (non-hiding) polynomial commitment schemes.

Definition 12. A polynomial commitment scheme PC is binding for Pgen , if for any $\lambda, n \in \text{poly}(\lambda)$ and PPT adversary $\mathcal{A}$, $\text{Adv}_{\text{Pgen}, \text{PC}, n, \mathcal{A}}^{\text{bind}}(\lambda) :=$

$$\Pr \left[C = \text{Com}(f_1) = \text{Com}(f_2) \wedge \left| \begin{array}{l} \mathbf{p} \leftarrow \text{Pgen}(1^\lambda); (\text{ck}, \text{tk}) \leftarrow \text{KGen}(\mathbf{p}, n); \\ (C, f_1, f_2) \leftarrow \mathcal{A}(\mathbf{p}, \text{ck}) \end{array} \right. \right] \approx_\lambda 0 .$$

Theorem 6. If a non-interactive PCS PC is black-box extractable and binding, it is non-adaptively evaluation-binding.

Proof. Let $(\mathcal{A}_{eb}, \text{P}_{eb}^*)$ be some arbitrary adversaries against non-adaptive evaluation-binding. We construct a binding adversary $\mathcal{A}_{\text{bind}}$ and black-box extractability adversaries $(\mathcal{B}_i, \text{P}_i^*)$, $i \in \{1, 2\}$, used by $\mathcal{A}_{\text{bind}}$. Since PC is black-box extractable, there exists an expected PPT extractor Ext_{bb} that works for $(\mathcal{B}_i, \text{P}_i^*)$, $i \in \{1, 2\}$.

We first describe $(\mathcal{B}_i, \text{P}_i^*)$, $i \in \{1, 2\}$, that are given (C, α) as an input. First, $\mathcal{B}_i(\mathbf{p}, \text{ck})$ is just the same as $\mathcal{A}_{eb}$, and returns (C, st) . Second, $\text{P}_i^*(\text{st}, \alpha)$ runs $\text{P}_{eb}^*(\text{st}, \alpha)$, obtaining either $(\eta_1 \neq \eta_2, \pi_1, \pi_2)$ or $\perp$. If P_{eb}^* succeeds in producing a collision, P_i^* returns $\text{tr}_i \leftarrow (C, \alpha, \eta_i, \pi_i)$. Otherwise, P_i^* returns $\perp$.

Since PC is black-box extractable, $\text{Ext}_{\text{bb}}^{\text{P}_i^*(\text{st}, \cdot)}(\text{ck}, \text{tr}_i)$ outputs a polynomial f_i , such that if $\mathbf{V}$ accepts tr_i then, with an overwhelming probability, $C = \text{Com}(f_i)$ and $f_i(\alpha) = \eta_i$.

Finally, $\mathcal{A}_{\text{bind}}(\text{ck})$ does the following:

1. Invoke $\mathcal{A}_{eb}(\text{ck}) = \mathcal{B}_1(\mathbf{p}, \text{ck}) = \mathcal{B}_2(\mathbf{p}, \text{ck})$, obtaining (C, st) .
2. Sample $\alpha \leftarrow \mathbb{F}$.
3. For $i \in \{1, 2\}$, run $\mathcal{B}_i(\text{ck}, C, \alpha)$ and $f_i \leftarrow \text{Ext}_{\text{bb}}^{\text{P}_i^*(\text{st}, \cdot)}(\text{ck}, \text{tr}_i)$ with P_i^* . Return $\perp$ if $f_1 = \perp$, $f_2 = \perp$, or $f_1 = f_2$.
4. Return (C, f_1, f_2) .

$\mathcal{A}_{\text{bind}}$ fails to break binding when,

- Extraction of either of the polynomials f_i fails, or
- $\mathcal{A}_{eb}$ fails to break evaluation binding.

Therefore,

$$1 - \text{Adv}_{\text{Pgen}, \text{PC}, n, \mathcal{A}_{\text{bind}}}^{\text{bind}}(\lambda) \leq \text{Adv}_{\text{Pgen}, \text{PC}, \text{Ext}, n, \mathcal{B}_1, \text{P}_1^*}^{\text{bbe}}(\lambda) + \text{Adv}_{\text{Pgen}, \text{PC}, \text{Ext}, n, \mathcal{B}_2, \text{P}_2^*}^{\text{bbe}}(\lambda) + (1 - \text{Adv}_{\text{Pgen}, \text{PC}, n, \mathcal{A}, \text{P}^*}^{\text{naeb}}(\lambda)) .$$

Consequently,

$$\begin{aligned} \text{Adv}_{\text{Pgen}, \text{PC}, n, \mathcal{A}_{\text{bind}}}^{\text{bind}}(\lambda) + \text{Adv}_{\text{Pgen}, \text{PC}, \text{Ext}, n, \mathcal{B}_1, \text{P}_1^*}^{\text{bbe}}(\lambda) + \text{Adv}_{\text{Pgen}, \text{PC}, \text{Ext}, n, \mathcal{B}_2, \text{P}_2^*}^{\text{bbe}}(\lambda) \\ > \text{Adv}_{\text{Pgen}, \text{PC}, n, \mathcal{A}, \text{P}^*}^{\text{naeb}}(\lambda) . \end{aligned}$$

However, note that $\mathcal{A}_{\text{bind}}$ is an expected PPT adversary, not a strict PPT adversary, as the binding definition requires. Here, we can apply Lemma 6. Since binding holds against all PPT adversaries, it also holds against all expected PPT adversaries.

Therefore, $\text{Adv}_{\text{Pgen}, \text{PC}, n, \mathcal{A}_{\text{bind}}}^{\text{bind}}(\lambda)$ is negligible and so are $\text{Adv}_{\text{Pgen}, \text{PC}, \text{Ext}, n, \mathcal{B}_i, \text{P}_i^*}^{\text{bbe}}(\lambda)$ for $i \in \{1, 2\}$ because BBE holds. It follows that $\text{Adv}_{\text{Pgen}, \text{PC}, n, \mathcal{A}, \text{P}^*}^{\text{naeb}}(\lambda)$ is negligible. $\square$